Agentic AI:

AI AGENTS FOR DEVELOPERS

1. WHAT ARE AI AGENTS?

→ AI agent = software system that can understand a goal, reason about it, and take actions.
→ It uses an LLM or reasoning engine plus tools, memory, and instructions.
→ Unlike a normal chatbot, an agent can decide next steps.
→ It can fetch data, call APIs, search docs, or trigger workflows.
→ Goal: move from answering to acting.

KEY IDEAS

- ✓ Goal-driven
- ✓ Reasoning
- ✓ Tool use
- ✓ Memory
- ✓ Action

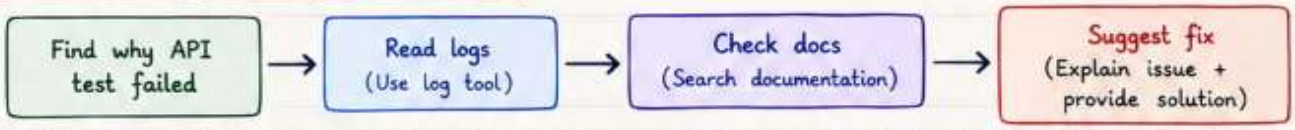
2. WHY DEVELOPERS SHOULD LEARN AGENTS?

- ✓ Build coding assistants and automation tools.
- ✓ Analyze logs, bugs, and test failures faster.
- ✓ Connect LLMs with APIs, DBs, and internal systems.
- ✓ Useful in backend, QA, DevOps, and support workflows.

3. BASIC AGENT FLOW



EXAMPLE FLOW: Find why API test failed



4. CHATBOT vs ASSISTANT vs AGENT

Type	What it does	Key Characteristics	Example Use Case
Chatbot	Mostly answers questions.	• Simple Q&A • No tool use (or very limited)	Answer general questions about a topic.
Assistant	Helps with tasks and context.	• Uses context and memory • Can follow instructions	Summarize a document, write code, or explain logs.
Agent	Plans, uses tools, and takes actions.	• Goal-driven • Can use multiple tools • Decides next steps	Find and fix a failed test by reading logs, searching docs, and suggesting a solution.

NOTE: A strong agent = clear goal + right context + right tools.

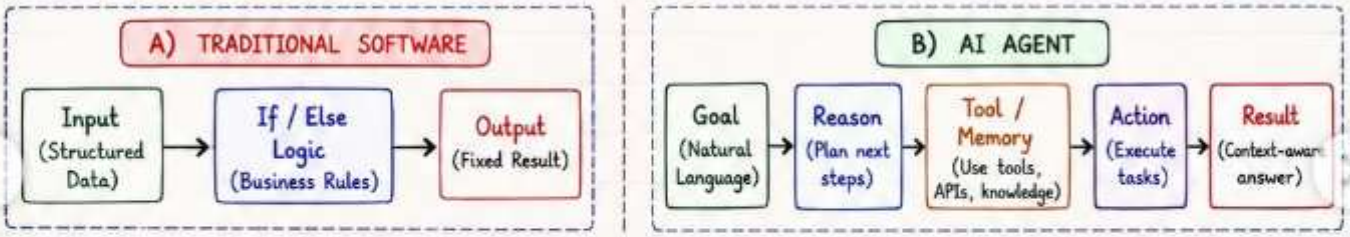
AI AGENTS FOR DEVELOPERS

1. AI AGENT vs TRADITIONAL SOFTWARE

→ Traditional software follows fixed logic; agents can reason and adapt before acting.

Feature	Traditional Software	AI Agent
Logic	Predefined rules / code	Dynamic reasoning + instructions
Input	Structured input	Natural language + context
Behavior	Fixed flow	Can plan next step
Tool Use	Hardcoded integration	Chooses tools when needed
Adaptability	Low	Higher
Output	Deterministic	Context-aware, flexible
Best For	Stable business rules	Knowledge tasks / decision support

2. HOW THEY WORK



3. WHEN TO USE WHICH?

Use traditional software when:

- Tasks have clear and well-defined rules.
- You need strict consistency and reliability.
- High-speed transactional logic (e.g., payments, order processing).

Use agents when:

- Tasks are ambiguous or involve open-ended questions.
- Need to search and use knowledge (documents, web, databases).
- Summarizing, analyzing or generating information.
- Multi-step tasks that require planning and tool usage.

4. REAL EXAMPLE

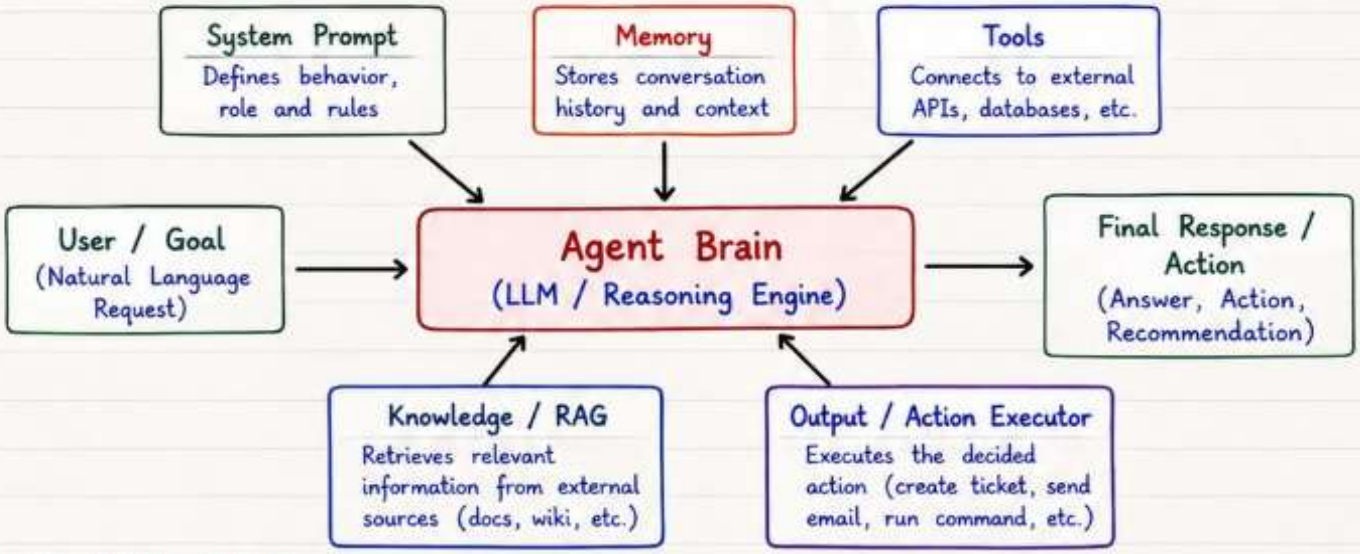
Task: Find failing order and notify customer.

Traditional Software Application	AI Agent Application
① Developer codes every step manually: • Query order from database / API • Check order status • If failed, get failure reason • Send email / notification to customer • Handle edge cases and errors	① Developer gives a high-level task to the agent. ② Agent reads the task and understands the goal. ③ Queries order API to find the failing order. ④ Checks the failure reason (using tools / APIs). ⑤ Drafts a friendly response and notifies the customer (via email / chat / notification).

★ **KEY TAKEAWAY:**
Agents do not replace software engineering; they add reasoning on top of software systems.

AI AGENTS FOR DEVELOPERS

3. CORE COMPONENTS OF AN AI AGENT



COMPONENTS TABLE

Component	Purpose	Example
Goal / Input	Defines the task for the agent.	"Summarize bug log"
Prompt	Sets behavior / rules for the agent.	System instructions (e.g. role, tone)
Memory	Stores conversation history and context.	Previous chat, preferences, user data
Tools	Allows the agent to take external actions.	Search API, database, email, etc.
Knowledge / RAG	Brings external facts and domain knowledge.	Docs, wiki, PDFs, codebase, etc.
Executor	Performs the final action decided by the agent.	Create ticket, send reply, run command

HOW THEY WORK TOGETHER

- ✓ Prompt controls behavior.
- ✓ LLM decides what to do next.
- ✓ Memory keeps session context.
- ✓ Tools fetch live data or perform actions.
- ✓ Retrieval improves factual answers.

DEVELOPER EXAMPLE

Task	:	Explain why payment API failed
Context	:	Logs + docs
Tool	:	Log search (search error logs)
Output	:	Root cause + suggested fix (e.g. invalid API key)

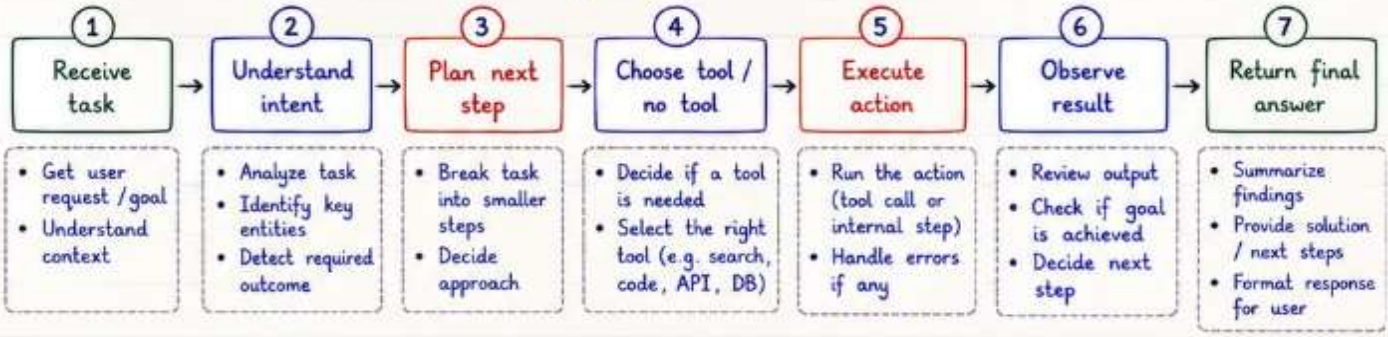
★ **IMPORTANT:**

A useful agent is not just an LLM; it is an LLM connected to context, tools, and actions.

AI AGENTS FOR DEVELOPERS

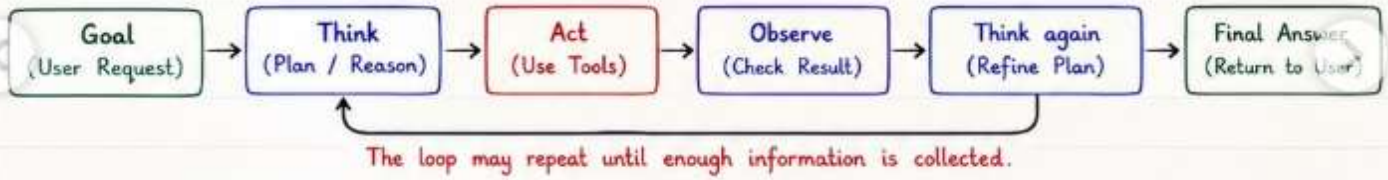
1. HOW AN AI AGENT WORKS

→ An AI agent takes a user goal, understands it, plans steps, uses tools when needed, executes actions, observes results and gives a final answer.



2. AGENT LOOP (THINK → ACT → OBSERVE)

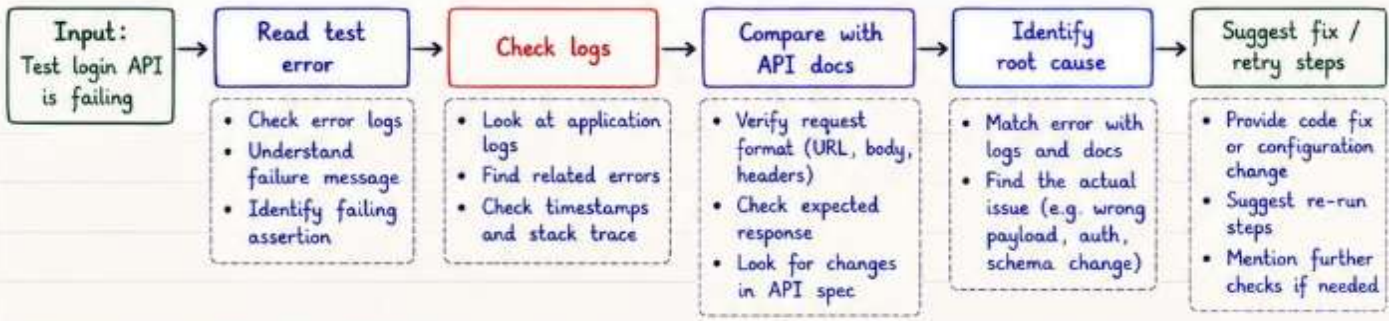
→ Agents work in a loop of thinking, taking action, observing results and repeating until they can provide a final answer.



The loop may repeat until enough information is collected.

3. PRACTICAL EXAMPLE: DEBUG FAILED API TEST

→ Example of how an AI agent can debug a failing API test using multiple steps and tools.



4. WHY THIS FLOW MATTERS

- ✓ Helps solve multi-step tasks.
- ✓ Reduces guessing.
- ✓ Makes tool usage systematic.
- ✓ Supports autonomous workflows.

☆ **TIP:**
Good agents reason in small steps and verify results before answering.

AI AGENTS FOR DEVELOPERS

1. PROMPTS, ROLES & INSTRUCTIONS

→ Prompts tell the agent what to do and how to behave.
→ A well-structured prompt improves consistency and quality.
→ Agents usually use multiple instruction layers.

KEY IDEA

Good prompts help agents understand the task, follow the right behavior and produce better results.

2. PROMPT LAYERS

Layer	Purpose	Example
System	Overall role / rules for the agent.	"You are a bug triage agent."
Developer	App-specific behavior and additional instructions.	"Return steps in bullet points."
User	Actual request or task to be completed.	"Analyze this error log."

3. GOOD PROMPT STRUCTURE

- ✓ Define role (who the agent is)
- ✓ Define goal (what it should achieve)
- ✓ Provide context (relevant details)
- ✓ Mention constraints (what to avoid / limits)
- ✓ Specify output format (structure of response)
- ✓ Add examples if needed (to guide the output)

4. SAMPLE SYSTEM PROMPT

```
1  You are an API support agent.
2  Help developers debug REST API issues.
3  Use tools when needed.
4  Return: probable cause, checks, and next steps.
5  Do not invent missing facts.
```

5. BAD vs BETTER

Bad Prompt	Better Prompt
"Help me fix this"	"Act as a backend support agent. Analyze the 500 error log and return likely cause + validation steps."

KEY POINT

Clear instructions reduce hallucination and make agents more reliable.

AI AGENTS FOR DEVELOPERS

1. TOOLS IN AI AGENTS

→ Tools let an agent interact with external systems.

→ Without tools, an LLM can mostly only generate text.

→ With tools, an agent can fetch live data and perform actions.

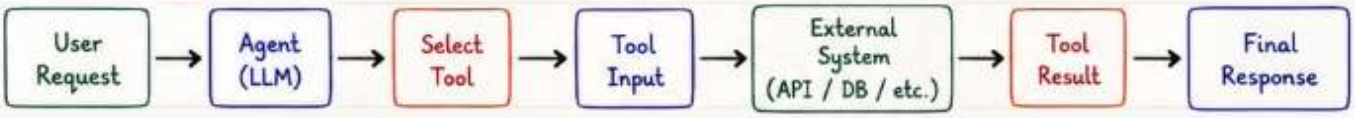
KEY POINT

Tools extend the capabilities of an AI agent beyond text generation by allowing it to interact with the real world.

2. COMMON TOOL TYPES

Tool Type	Use Case	Example
Search / Retrieval	Find knowledge from external sources or documents.	Search docs, wiki, knowledge base
API Tool	Fetch live data from external APIs.	Weather, payments, tickets
Database Tool	Query records from a database.	SQL query, order lookup
Code / Calculator	Compute, execute code or perform calculations.	Math operations, regex, code sandbox
Action Tool	Trigger actions or workflows.	Send email, create Jira ticket

3. TOOL CALLING FLOW



4. PRACTICAL EXAMPLE

```
Tool: get_order_status(orderId)
Input: 10245
Output:
{
  status: "FAILED",
  reason: "Payment timeout"
}
```

→

Agent's Final Response

"Order 10245 failed due to payment timeout."

5. WHEN SHOULD AN AGENT USE A TOOL?

- ✓ When answer needs fresh or real data.
- ✓ When action must be performed.
- ✓ When calculation / lookup is required.
- ✓ When user asks for system-specific information.

★ **NOTE:**

Tool quality matters — clear inputs, reliable outputs, and safe permissions are essential.

AI AGENTS FOR DEVELOPERS

8. TOOL CALLING & ACTION EXECUTION

1. INTRODUCTION TO TOOL CALLING

→ Tools allow agents to access external systems like files, databases, APIs, and development tools.

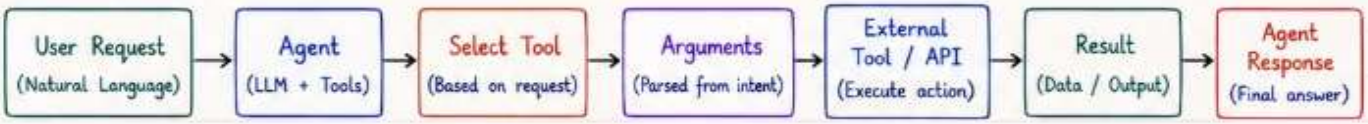
→ Without tools, an LLM can only generate text and cannot interact with the real world.

→ Tools help agents fetch data, run commands and take real actions on your behalf.

KEY POINT

Tool calling gives LLMs the ability to go beyond text generation and actually interact with external systems.

TOOL CALLING FLOW



COMMON TOOLS FOR DEV AGENTS

Tool	Purpose	Example
file_reader	Read files and code from the project.	Open and read service.py
shell_runner	Execute shell commands in the development environment.	Run pytest -q
search_docs	Search documentation (internal or external).	Find API usage for FastAPI or Spring Boot
git_tool	Inspect and interact with git repository.	Show diff, list branches, or create a commit

EXAMPLE TOOL SCHEMA

```
{
  "name": "run_tests",
  "description": "Run unit tests for a specific module",
  "args": {
    "module": "auth"
  }
}
```

MINI EXAMPLE (STEP BY STEP)

1. User Request — "Run unit tests for auth module"
2. Agent Chooses Tool — Selects run_tests tool with module = "auth"
3. Tool Execution — Agent calls run_tests(module="auth") and reads the output.
4. Agent Response — Summarizes test results, highlights failures and suggests next steps.

KEY BENEFITS

- ✓ Real data access (files, databases, APIs)
- ✓ Automation of repetitive tasks
- ✓ Repeatable and consistent actions
- ✓ Better and more accurate responses
- ✓ Extends LLM capabilities to real-world use cases

CAUTION / SAFETY

- ⚠ Validate tool inputs to prevent unsafe commands.
- ⚠ Restrict tool permissions (principle of least privilege).
- ⚠ Confirm risky actions (e.g., delete, deploy, write).
- ⚠ Use sandboxed environments for command execution.

AI AGENTS FOR DEVELOPERS

9. MEMORY IN AI AGENTS

→ Memory helps AI agents keep context across multiple steps and interactions.

→ It is essential for multi-step tasks like code generation, debugging and project assistance.

→ It improves personalization and continuity, making the agent more useful over time.

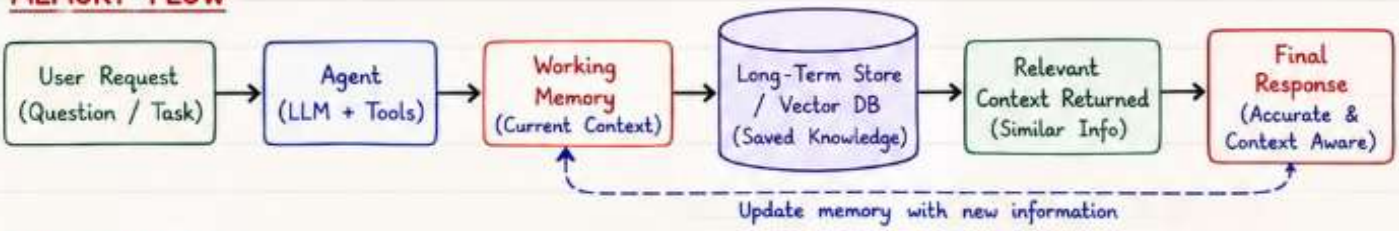
KEY POINT

Memory allows agents to remember past information, user preferences and project context, so they can give more relevant and consistent responses.

TYPES OF MEMORY

Type	Stores	Example
Working Memory	Current task context (active information).	Active TODO list, current code file, ongoing task details.
Short-Term Memory	Recent conversation history.	Last user request, recent messages in the same session.
Long-Term Memory	Saved knowledge, preferences and project facts.	Coding preferences, project structure, important decisions, tech stack.
Vector Memory	Embeddings of documents, code or text.	Retrieve similar docs, past code examples, error solutions.

MEMORY FLOW



DEV EXAMPLES

- ✓ Remember project repository path and structure.
- ✓ Recall testing framework used in the project.
- ✓ Store and follow coding style and conventions.
- ✓ Learn from common failure patterns and past errors.

MEMORY ENTRY EXAMPLE

```
user_pref = {
    "language": "Java",
    "test_framework": "JUnit"
}
```

WHEN TO USE WHICH

- Use Working Memory for current execution context and active task details.
- Use Long-Term Memory for user preferences, project facts and important knowledge.
- Use Vector Memory for document retrieval, code search and finding similar information.

⚠ CAUTION

- Stale or incorrect memory can cause wrong or irrelevant actions.
- Summarize and prune memory regularly to keep it accurate and useful.

10. RAG FOR DEVELOPER AGENTS

→ RAG stands for Retrieval-Augmented Generation.

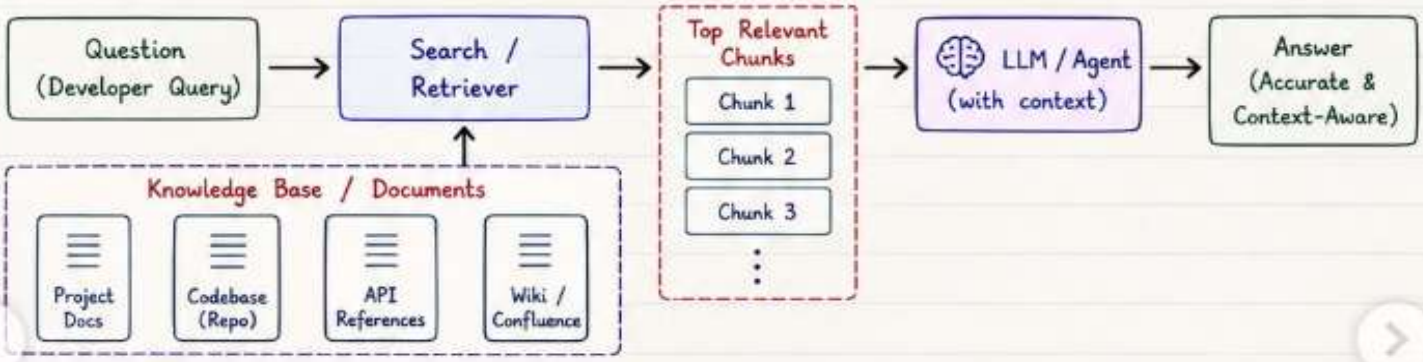
→ The agent retrieves relevant knowledge from external sources (docs, codebase, wikis, etc.) before answering.

→ Useful when project documentation or codebase details matter for accurate and up-to-date responses.

KEY POINT

RAG helps developer agents give accurate, context-aware answers using your own documents and code.

RAG FLOW



WHY DEV AGENTS NEED RAG?

- ✓ Access to internal project documentation.
- ✓ Look up API references and usage examples.
- ✓ Understand codebase structure and specific modules.
- ✓ Find runbooks, troubleshooting guides and best practices.
- ✓ Help with onboarding and answering project-specific questions.

RAG PIPELINE (5 STEPS)

1. Collect documents (docs, code, wikis, etc.).
2. Chunk text into smaller pieces (e.g. 500-1000 tokens).
3. Create embeddings for each chunk.
4. Store embeddings in a vector database.
5. Retrieve relevant chunks and generate answer using LLM / agent.

GOOD SOURCES FOR RAG

Source	Useful For
Documentation	Product docs, internal docs, design docs
Code comments	Function/class explanations, usage examples
Wiki / Confluence	Architecture notes, onboarding guides
Past tickets	Real issues, solutions, edge cases

EXAMPLE (SIMPLE RAG USAGE)

```
# Developer query
query = "How does payment retry work?"
# Retrieve relevant documents
retrieved = search_docs(query)
# Generate answer using agent with context
answer = agent(query, context=retrieved)
print(answer)
```

☆ **TIP:** Good chunking (right size) and clean metadata (file name, section, tags, etc.) help retrieve more relevant information and improve answer quality.

11. MULTI-AGENT SYSTEMS

→ Sometimes a single agent is not enough to handle complex tasks.

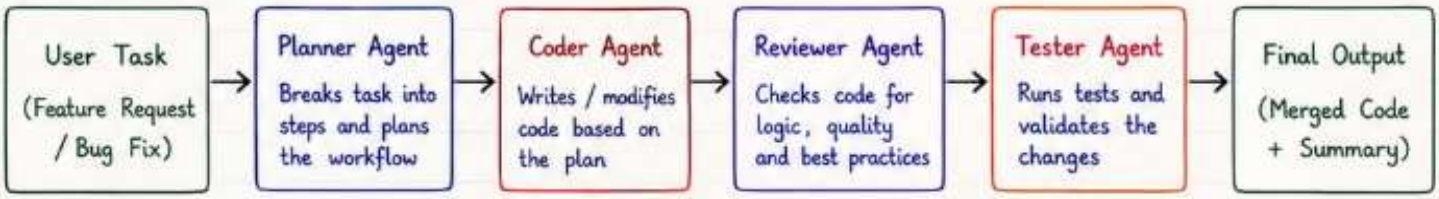
→ Multiple specialized agents can collaborate, each handling a specific role.

→ This approach helps in larger workflows like code changes, repository analysis, testing and feature development.

KEY POINT

Multi-agent systems combine the strengths of multiple specialized agents to solve complex tasks more effectively.

EXAMPLE MULTI-AGENT FLOW



WHEN TO USE?

- ✓ Large code changes with multiple files.
- ✓ Repository-wide analysis and refactoring.
- ✓ Test + review workflows.
- ✓ Research + implementation tasks (e.g., new feature development).

SINGLE AGENT vs MULTI-AGENT

Aspect	Single Agent	Multi-Agent
Simplicity	Simple to set up and use.	More complex to set up.
Specialization	Handles all tasks generally.	Each agent has specific expertise.
Coordination	Easy (no coordination needed).	Harder (requires agent coordination).
Use Case	Small and simple tasks.	Larger and complex workflows.

ROLE EXAMPLES

Agent	Role / Description
Planner Agent	Breaks down the user task into smaller steps and creates a plan.
Coder Agent	Writes or modifies code based on the plan.
Reviewer Agent	Reviews the code for correctness, logic, security and best practices.
Tester Agent	Runs tests (unit/integration) and validates that the changes work.

SAMPLE WORKFLOW (PSEUDO CODE)

```
# Multi-agent workflow
plan = planner_agent.create_plan(user_task)
code = coder_agent.implement(plan)
review = reviewer_agent.review(code)
test_result = tester_agent.run_tests(code)

if test_result.passed:
    merge_summary = generate_summary(code, review)
    print("✅ Code ready to merge!")
else:
    print("❌ Tests failed. Please check and fix.")
```

☆ **NOTE:**

Having too many agents can increase latency and coordination overhead. It's important to design the right number of agents for your use case.

→ Use multiple agents when the task is complex enough to need specialization.

12. PRACTICAL DEV AGENT WORKFLOW

→ A developer agent can read a GitHub issue, inspect relevant files, suggest a patch, run tests and summarize the results.

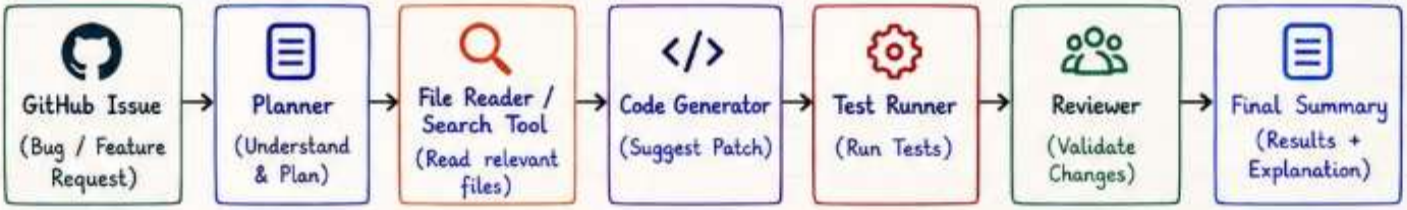
→ It uses LLM + tools to understand the codebase and perform real development tasks.

→ Helps developers fix bugs faster and maintain code with less manual effort.

KEY POINT

- Developer agents act like a pair programmer.
- They can read, understand, modify, test and explain code.
- Always keep human review before merging changes.

END-TO-END FLOW



STEP-BY-STEP WORKFLOW

1. **Read Issue** - Understand the problem from GitHub issue (description, logs, repro steps).
2. **Inspect Code** - Search and read relevant files using tools (file search, code navigation).
3. **Create Plan** - Decide what needs to be changed and which files to modify.
4. **Edit Code** - Generate and apply patch to fix the issue.
5. **Run Tests** - Execute relevant tests to verify the fix works.
6. **Return Explanation** - Summarize changes, test results and reasoning.

SAMPLE CODE (CONCEPT)

```
# Bug Fix Agent - Example
issue = "Login fails for empty password"
files = read_repo("auth/")              # read relevant files
patch = agent.fix(issue, files)         # generate patch
result = run_tests("auth")              # run tests

print("Patch Suggestion:")
print(patch)                            # diff / modified files
print("Test Status:", result.status)    # pass / fail
print("Summary:", result.explanation)   # reasoning
```

EXPECTED OUTPUT

- ✓ Patch suggestion (diff / code changes)
- ✓ List of changed files
- ✓ Test status (pass / fail)
- ✓ Reasoning summary (what was fixed and why)

REAL USE CASES

- Bug fixing (e.g., fix login, handle edge cases)
- Code review (find issues and suggest improvements)
- Refactoring (improve code quality, remove duplication)
- PR description generation (summarize changes automatically)

PRO TIP

Always keep human review before merging agent-generated changes into production.

13. STRUCTURED OUTPUTS FOR AGENTS

→ Structured outputs make agent responses predictable, machine-readable and easier to validate.

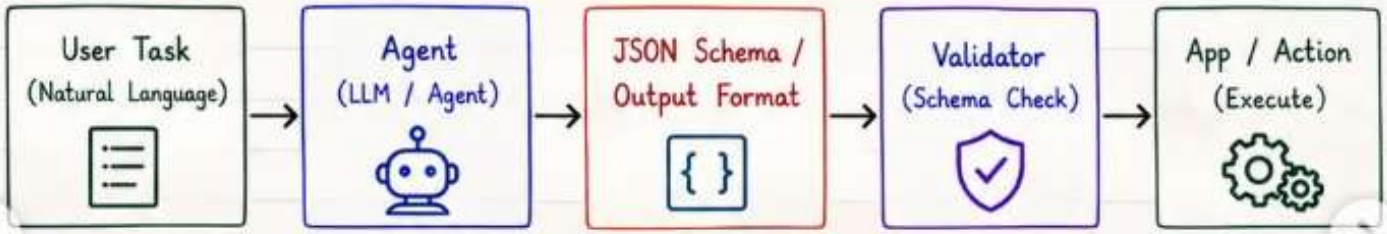
→ They ensure the agent returns data in a defined format instead of free-form text.

→ Useful for integrating agents with tools, APIs and automated workflows.

KEY POINT

Structured outputs turn unpredictable agent responses into reliable, machine-readable results that can be validated and used in real applications.

HOW STRUCTURED OUTPUTS WORK (HIGH LEVEL FLOW)



WHY USE STRUCTURED OUTPUTS?

- ✓ Reliable parsing of agent responses
- ✓ Fewer integration errors
- ✓ Easier automation and tool usage
- ✓ Safer and more controlled downstream actions

COMMON OUTPUT FIELDS (EXAMPLE)

Field	Type	Example
task	string	code_review
priority	string	high
requires_human	boolean	true
summary	string	fix null check

EXAMPLE - STRUCTURED OUTPUT (JSON)

```
{
    "task": "bug_fix",
    "priority": "high",
    "requires_human": true,
    "summary": "Fix null pointer in payment service"
}
```

TIP

Combine structured outputs with schema validation before executing actions.

14. GUARDRAILS, RETRIES & ERROR HANDLING

→ Guardrails keep agents safe, reliable and predictable.

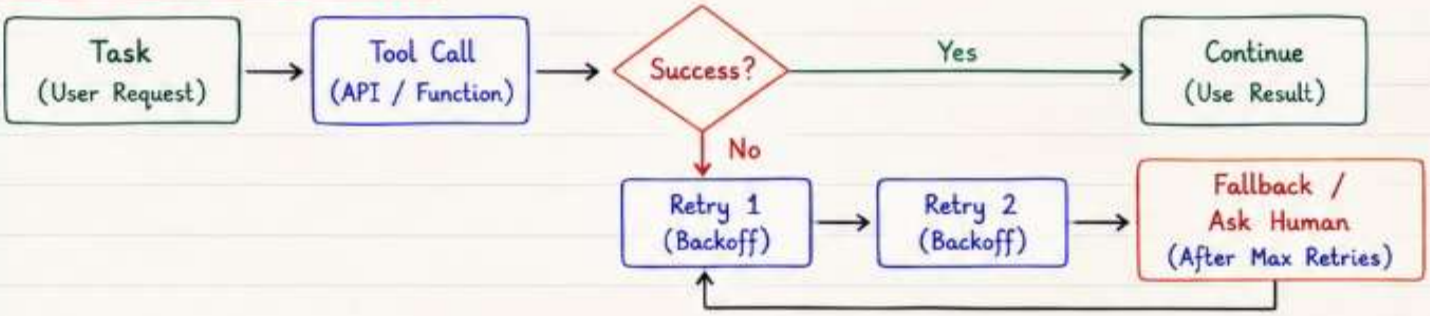
→ They restrict unsafe actions, validate inputs and help avoid bad outputs.

→ Retries and error handling make agents resilient to real world failures (API errors, invalid outputs, etc).

KEY POINT

Good agents don't just work when things go right, they handle failures gracefully and safely.

RETRY FLOW (HIGH LEVEL)



COMMON GUARDRAILS

- ✓ Allowed tools list – Only let the agent use safe and approved tools.
- ✓ Max retries – Limit number of retries to avoid infinite loops.
- ✓ Schema validation – Validate tool inputs and model outputs (e.g., JSON schema).
- ✓ Blocked commands – Prevent dangerous operations (e.g., delete, rm -rf, system calls).
- ✓ Confidence checks – Verify model confidence before taking important actions.
- ✓ Rate limits – Avoid excessive API calls and control usage costs.

COMMON ERRORS & AGENT RESPONSES

Error	Agent Response
API timeout	Retry with exponential backoff (e.g., 2 retries).
Invalid JSON output	Regenerate with clearer instructions / stricter schema.
Missing required data	Ask user for clarification (provide what's missing).
Rate limit error	Wait and retry (with backoff) or use cached result.
Risky or unsafe action	Block and require human approval.

SIMPLE PSEUDO-CODE

```
try:
    result = tool_call(input)                          # Call the tool
except TimeoutError:                                   # Retry on timeout
    result = retry(tool_call, retries=2, backoff=True)
except InvalidOutputError:                             # Regenerate on invalid output
    result = regenerate(input, schema=expected_schema)

if is_risky_action(result):
    require_approval(result)                           # Human approval for risky actions
else:
    return result                                      # Use the result
```

TIP

Good agents fail safely, not silently.

15. HUMAN-IN-THE-LOOP (HITL)

→ Human-in-the-loop (HITL) means the agent asks for approval or review before important actions.

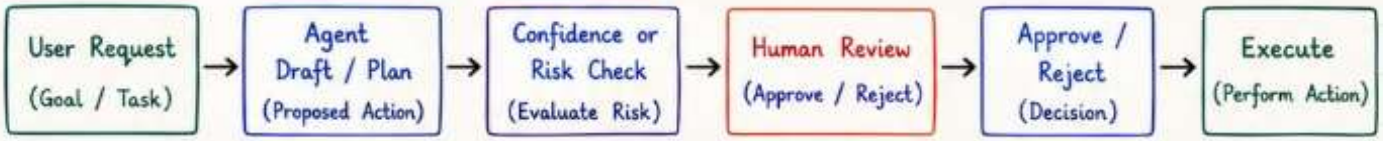
→ It combines the speed of automation with human judgment and domain knowledge.

→ Helps prevent mistakes, unsafe actions and unexpected results in real-world applications.

KEY POINT

HITL keeps a human in the decision loop for sensitive or high-impact actions.

HUMAN-IN-THE-LOOP FLOW (HIGH LEVEL)



WHEN HITL IS USEFUL?

- ✓ Deployments to production
- ✓ Code merge (PR approval)
- ✓ Payments or refunds
- ✓ Emails to customers
- ✓ Deleting data
- ✓ Access or permission changes

LOW RISK vs HIGH RISK EXAMPLES

Low Risk	High Risk
Read files and search code	Deploy to production
Generate a code snippet	Merge code to main branch
Run tests or lint checks	Process payments or refunds
Create a draft email (internal)	Send emails to customers
Create documentation	Delete data or drop tables
Analyze logs and suggest fix	Change user roles or permissions

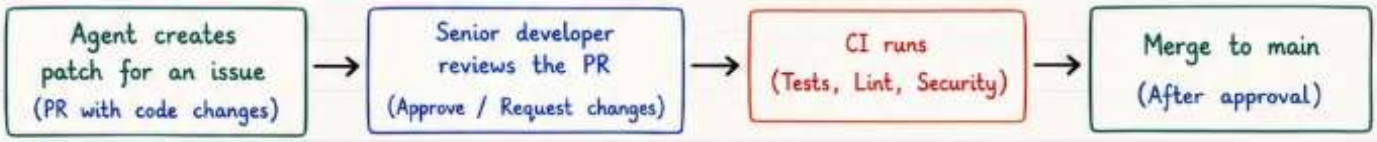
SIMPLE PSEUDO-CODE

```
# agent decides what to do based on risk
if risk_score > threshold:
    ask_human()   # wait for approval
else:
    execute()     # perform the action
```

NOTE

HITL improves safety, trust, and auditability.

REAL-WORLD EXAMPLE – PR ASSISTANT



16. BUILDING A CODING AGENT

→ A coding agent can read repository files, search code, suggest edits, run tests and summarize changes.

→ It uses tools to interact with your codebase safely.

→ Helps automate repetitive development tasks and speeds up the development process.

KEY POINT

A coding agent understands your task, uses tools to work with the codebase and gives a clear summary of changes.

CODING AGENT ARCHITECTURE (HIGH LEVEL)



TYPICAL WORKFLOW

1. **Understand the task**
   Read the developer prompt and identify what needs to be changed.
2. **Inspect the repository**
   Search and read relevant files to understand the current code.
3. **Propose the change**
   Plan the edits and show what will be modified.
4. **Apply the patch**
   Modify the files using tools (e.g., edit_file).
5. **Run tests and explain**
   Run tests to verify the change and summarize the results.

SAMPLE AGENT LOOP (PSEUDOCODE)

```
# task: natural language request
task = "Add validation to signup API"

# plan the steps
plan = agent.plan(task)

# find relevant files
files = search_code(plan)

# apply changes
patch = edit_file(files)

# run tests to verify
result = run_tests()

# return final summary
return summarize(patch, result)
```

TOOLS USED BY CODING AGENT

Tool	Why used
search_code	Find relevant files, functions or classes.
read_file	Read file contents to understand existing code.
edit_file	Apply code changes or create new files.
run_tests	Run unit/integration tests to verify the changes.
git_diff	Show the changes made in a readable diff format.

NOTE

Best for bug fixes, refactors, tests, and documentation updates.

EXAMPLE USE CASE – ADD VALIDATION TO SIGNUP API

1. Developer Prompt

"Add email and password validation to the signup API and include tests."

→

2. Agent Actions

- Search for signup endpoint files
- Read existing validation logic
- Add input validation
- Run tests
- Show git diff and summary

→

3. Result

- Updated signup API with validation
- Added/updated test cases
- All tests passing
- Summary of changes (files modified, what was added and why)

17. DEBUGGING & PR REVIEW AGENTS

→ Debugging agents analyze logs, stack traces, tests and code diffs to find the root cause of an issue.

→ PR review agents check code quality, security risks and potential issues in pull requests.

→ They help developers fix bugs faster and maintain high quality code.

KEY POINT

Debugging and PR review agents act as an AI pair programmer to find issues early, suggest fixes and improve code quality.

DEBUGGING FLOW (HOW IT WORKS)



PR REVIEW FLOW (HOW IT WORKS)



WHAT THE AGENT SHOULD CHECK?

- ✓ Failing tests and test coverage
- ✓ Null pointer / runtime errors
- ✓ API contract changes (breaking changes)
- ✓ Security issues (auth, injection, secrets)
- ✓ Performance regressions (slow queries, high CPU)
- ✓ Missing documentation or unclear logic

COMMON SIGNALS AND POSSIBLE ISSUES

Signal	Possible Issue
500 errors in logs	Backend bug / unhandled exception
NullPointerException	Null value not handled
Failing auth test	Token / configuration issue
Slow database query	DB performance issue
Large code diff	High risk change
Secrets in diff	Security risk (e.g. API key)

EXAMPLE – SIMPLE DEBUG / REVIEW AGENT

```
# Read logs and code diff, analyze and return summary
logs = read_logs()
diff = git_diff()
issues = analyze(logs, diff)
return review_summary(issues)
```

NOTE:

Always keep human review before merging important fixes. Agents help find issues and suggest changes, but final decision should be made by a developer.

18. EVALUATION, OBSERVABILITY & PRODUCTION CHECKLIST

→ Production agents must be measurable, observable and reliable.

→ Evaluation helps us understand how well the agent is performing.

→ Observability helps us debug issues, improve behavior and ensure safe operation in production.

KEY POINT

You can't improve what you can't measure. Evaluation and observability make agents reliable in real-world use.

AGENT OBSERVABILITY FLOW



KEY EVALUATION METRICS

- ✓ **Task success rate**
  (How often the agent completes the goal)
- ✓ **Tool success rate**
  (How often tool calls succeed)
- ✓ **Latency**
  (Time taken to complete the task)
- ✓ **Token cost**
  (Cost per task or per successful run)
- ✓ **Hallucination rate**
  (Incorrect or made-up information)
- ✓ **Human approval rate**
  (How often humans approve the output)

WHAT TO MONITOR?

What to monitor	Why	Example
Traces	Debug failures and understand behavior.	Failed tool chain, wrong reasoning step
Tool calls	Ensure tools work reliably.	API errors, timeout, invalid arguments
Latency	Track performance and user experience.	Average time per task
Token cost	Control spend and optimize prompts.	Tokens per task, cost per successful run
Errors / Failures	Identify and fix issues quickly.	Exception logs, failed runs
User feedback	Measure real-world usefulness.	Thumbs up/down, manual review

PRODUCTION CHECKLIST

- ✓ Use structured outputs (JSON / schema)
- ✓ Implement retries for transient failures
- ✓ Add guardrails and safety checks
- ✓ Use human approval for critical tasks
- ✓ Define memory and data retention policy
- ✓ Set up monitoring (logs, metrics, alerts)
- ✓ Create a feedback loop to improve performance

SIMPLE EVALUATION SNIPPET

```
# Evaluate agent output
score = evaluate(task, output)
log(metrics)
if score < threshold:
    improve_prompt()
# Retry with better prompt or strategy
```

FINAL TAKEAWAY

★ Good agents are not just smart — they are safe, observable, and useful.

AGENTIC AI PROJECT

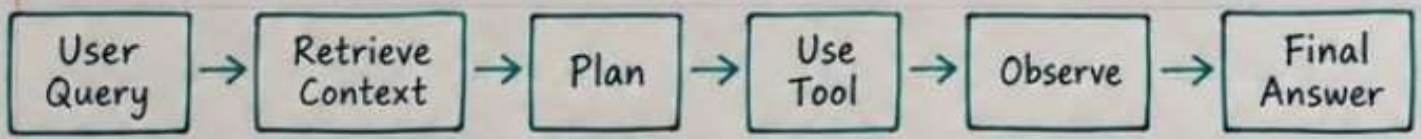
PROJECT FILE STRUCTURE

```
agentic-ai-project/
├── src/agent_app/
│   ├── config.py
│   ├── llm_client.py
│   ├── state.py
│   ├── planner.py
│   ├── tools.py
│   ├── executor.py
│   ├── memory.py
│   ├── rag.py
│   ├── agent.py
│   └── api.py
├── data/knowledge/
├── tests/
│   ├── test_tools.py
│   └── test_agent.py
├── .env.example
├── requirements.txt
└── main.py
```

What Each Part Does

config	→	environment & limits
llm_client	→	model requests
state	→	shared agent state
planner	→	creates next steps
tools	→	safe external actions
executor	→	runs approved tools
memory	→	conversation history
rag	→	private knowledge search
agent	→	observe-think-act loop
api	→	HTTP endpoints
tests	→	deterministic checks

CORE FLOW



Build the smallest reliable loop first; add memory and RAG only when needed.

CONFIGURATION & MODEL CLIENT

① config.py — Load Settings Safely

```python
from dataclasses import dataclass
import os

@dataclass
class Settings:
    api_key: str = os.getenv("ANTHROPIC_API_KEY", "")
    model: str = os.getenv("MODEL_NAME", "claude-sonnet-4-5")
    max_steps: int = int(os.getenv("MAX_STEPS", "6"))
    timeout_s: int = 30

settings = Settings()
if not settings.api_key:
    raise RuntimeError("Missing API key")
```

→ Keep secrets in environment variables — not source code.

② llm_client.py — One Model Gateway

```python
from anthropic import Anthropic
from .config import settings

client = Anthropic(api_key=settings.api_key)

def ask_model(messages, tools=None):
    return client.messages.create(
        model=settings.model,
        max_tokens=1200,
        messages=messages,
        tools=tools or []
    )
```

WHY ONE GATEWAY?

→ Central place for retries, timeout, logging, model changes and cost tracking.

STATE & PLANNING

① state.py — Shared Agent State

```python
from dataclasses import dataclass, field
from typing import Any

@dataclass
class AgentState:
    query: str
    messages: list[dict] = field(default_factory=list)
    plan: list[str] = field(default_factory=list)
    observations: list[Any] = field(default_factory=list)
    step: int = 0
    done: bool = False
```

query → plan → action → observation → answer

② planner.py — Convert Goal into Steps

```python
from .llm_client import ask_model

def create_plan(state: AgentState) -> list[str]:
    prompt = f"""
        Goal: {state.query}
        Return 3-5 short executable steps.
        Use tools only when required.
    """
    response = ask_model([
        {"role": "user", "content": prompt}
    ])
    return parse_steps(response)
```

① State prevents hidden data flow.

② Plans must be short, observable and revisable.

③ Never execute raw model text directly.

KEY TAKEAWAY: The planner suggests steps; policy and code decide what may run.

TOOLS & SAFE EXECUTION

① tools.py — Define Allowed Actions

```python
from typing import Callable

def get_weather(city: str) -> dict:
    return {"city": city, "temp_c": 28}

TOOL_REGISTRY: dict[str, Callable] = {
    "get_weather": get_weather
}

TOOL_SCHEMAS = [{
    "name": "get_weather",
    "description": "Get current weather by city",
    "input_schema": {
        "type": "object",
        "properties": {"city": {"type": "string"}},
        "required": ["city"]
    }
}]
```

② executor.py — Validate Before Running

```python
def execute_tool(name: str, args: dict):
    if name not in TOOL_REGISTRY:
        raise ValueError("Tool not allowed")
    if not isinstance(args, dict):
        raise TypeError("Arguments must be an object")
    return TOOL_REGISTRY[name](**args)
```

Safety Checklist

- ☑ allow-list tools
- ☑ validate arguments
- ☑ set timeout
- ☑ redact secrets
- ☑ require approval for risky actions

The model chooses a tool; trusted application code executes it.

CONVERSATION MEMORY

① memory.py — Bounded Session Memory

```python
from collections import deque

class Memory:
    def __init__(self, limit: int = 12):
        self.items = deque(maxlen=limit)

    def add(self, role: str, content: str):
        self.items.append({
            "role": role,
            "content": content
        })

    def messages(self) -> list[dict]:
        return list(self.items)

    def clear(self):
        self.items.clear()
```

Example

```python
memory = Memory(limit=12)

memory.add("user",
        "Book a meeting tomorrow")

memory.add("assistant",
        "What time works?")
```

② What Should Be Remembered?

Keep	Avoid
user preferences	passwords / API keys
task decisions	raw private data
short summaries	entire long chats
tool results needed later	duplicate observations

③ Memory Flow in an Agent

Recent messages → Summarize old context → Store useful facts → Build next prompt

④ Notes

- Conversation memory is not model training.
- Add retention and deletion rules.
- Retrieve only relevant memories.

Bottom Takeaway: Good memory is selective, scoped and disposable.

RAG — USE YOUR OWN DATA

① rag.py — Ingest Knowledge

```
def ingest (documents, embedder, vector_db):
    chunks = []
    for doc in documents:
        chunks += split_text (
            doc,
            chunk_size=600,
            overlap=80
        )
    vectors = embedder.embed (chunks)
    vector_db.upsert (vectors, chunks)
```

```
# Split documents into chunks
# chunk_size: tokens per chunk
# overlap: keep context between
# adjacent chunks

# Convert chunks to embeddings
# Store in vector database
# Save both vectors and original text
```

② rag.py — Retrieve Context

```
def retrieve (query, embedder, vector_db, k=4):
    query_vector = embedder.embed_query (query)
    matches = vector_db.search (query_vector, top_k=k)
    return [
        {"text": m.text, "source": m.source}
        for m in matches
    ]
```

```
# Embed the user query
# Search for similar chunks
# Return top-k relevant chunks
# Include text and source metadata
```

Ingestion Pipeline: Files (Docs, PDFs, Web, DB) → Parse (Extract text) → Chunk (Split into segments) → Embed (Generate vectors) → Vector DB (Store vectors + metadata)

Query Pipeline: Query (User question) → Embed (Generate vector) → Similarity Search (Find nearest chunks) → Top Chunks (Relevant context) → Agent (Use context to answer)

Prompt Context

- Use only the supplied context.
- Cite the source.
- Say when evidence is missing.

Key Notes

- Chunk by meaning, not only length.
- Store source metadata.
- Evaluate retrieval separately.

Examples of Metadata

- source (file name / URL)
- page number / section
- title / date / author
- document type

Takeaway: RAG gives the agent relevant evidence — it does not guarantee correctness.

THE AGENT LOOP

agent.py — Observe, Decide, Act

```python
from .llm_client import ask_model
from .executor import execute_tool
from .rag import retrieve

def run_agent(query: str, max_steps: int = 6):
    messages = [{"role": "user", "content": query}]
    context = retrieve(query)

    for step in range(max_steps):
        response = ask_model(
            messages + [{"role": "user",
                         "content": f"Context: {context}"}],
            tools=TOOL_SCHEMAS
        )

        if response.stop_reason == "tool_use":
            call = get_tool_call(response)
            result = execute_tool(call.name, call.input)
            messages += tool_result_messages(call, result)
            continue

        return get_text(response)

    return "Stopped: step limit reached"
```

1 Load relevant context.
Use RAG (or other retrieval) to fetch background information for the query.

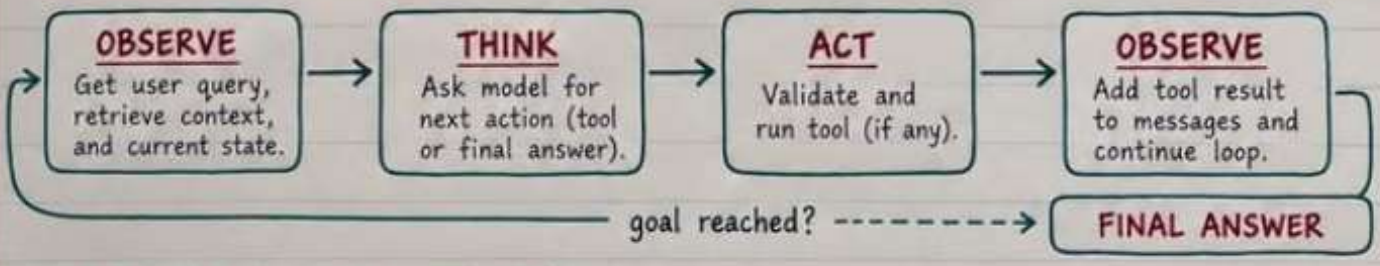
2 Ask model for next action.
Send conversation + context to the model with available tools (tool schemas). Model returns either a tool call or a final answer.

3 Validate and run tool.
If the model wants to use a tool, validate the call (name, args, allow-list) and execute it.

4 Add observation to messages.
Convert the tool result into messages and add to the conversation so the model can see what happened.

5 Stop on final answer or limit.
Return the model's final text answer, or stop if max steps is reached.

THE AGENT LOOP (VISUAL)



GUARDRAILS (KEEP IT CONTROLLED)

max steps | tool allow-list | timeout | user approval | trace ID

KEY TAKEAWAY

An agent is a controlled loop — not an unlimited autonomous process.

API ENTRY POINT

① api.py — Expose the Agent

```python
from fastapi import FastAPI
from pydantic import BaseModel
from .agent import run_agent

app = FastAPI(title="Agentic AI API")

class AskRequest(BaseModel):
    query: str

class AskResponse(BaseModel):
    answer: str

@app.post("/ask", response_model=AskResponse)
def ask(request: AskRequest):
    answer = run_agent(request.query)
    return AskResponse(answer=answer)

@app.get("/health")
def health():
    return {"status": "ok"}
```

② main.py — Start Locally

```python
import uvicorn

if __name__ == "__main__":
    uvicorn.run(
        "src.agent_app.api:app",
        host="0.0.0.0",
        port=8000,
        reload=True
    )
```

REQUEST FLOW

POST /ask → Validate Input → Run Agent → Return Answer

EXAMPLE

Request:	Response:
{"query": "What is the weather in Delhi?"}	{"answer": "Delhi is currently 28°C."}

PRODUCTION CHECKLIST

- ☑ authentication
- ☑ rate limit
- ☑ request ID
- ☑ CORS policy
- ☑ safe error messages

KEY TAKEAWAY

Keep transport logic thin; the agent remains testable outside the API.

TESTING, EVALUATION & RUN

① test_tools.py — Deterministic Unit Test

```python
def test_weather_tool():
    result = get_weather("Delhi")
    assert result["city"] == "Delhi"
    assert "temp_c" in result
```

② test_agent.py — Stub the Model

```python
def test_agent_stops_on_final_answer(monkeypatch):
    fake = FakeResponse(
        text="Task complete",
        stop_reason="end_turn"
    )
    monkeypatch.setattr(
        "agent_app.agent.ask_model",
        lambda *args, **kwargs: fake
    )
    assert run_agent("Say done") == "Task complete"
```

③ Evaluate More Than Accuracy

Metric	Question
task success	Was the goal completed?
tool accuracy	Was the correct tool used?
grounding	Is the answer supported?
safety	Was risky action blocked?
latency	How long did it take?
cost	How many tokens/tools?

④ Run the Project

```
pip install -r requirements.txt
export ANTHROPIC_API_KEY="..."
python main.py
pytest -q
```

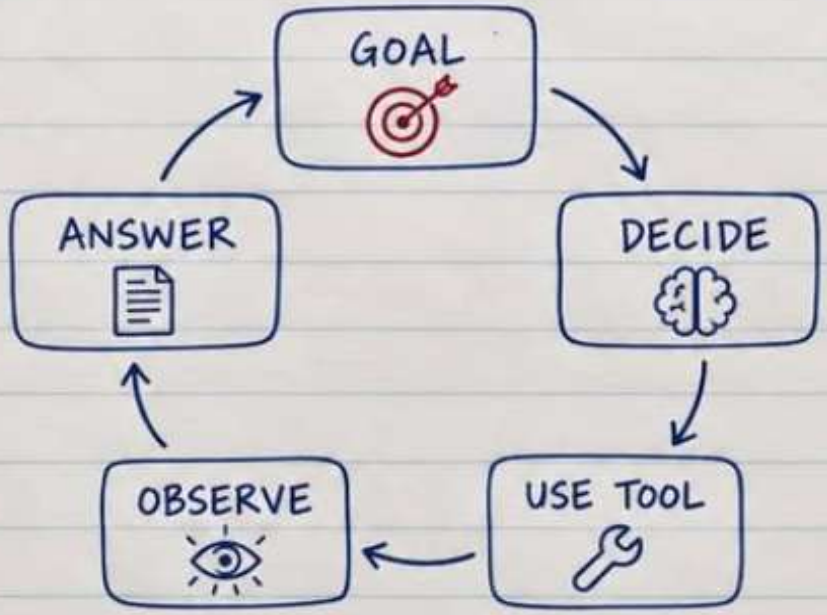
BUILD ORDER

1. deterministic tools
2. model gateway
3. bounded agent loop
4. tests and traces
5. memory and RAG

Start simple.
Measure every step.
Grant the minimum permission.

TOP 5 BEGINNER AGENTIC AI PROJECTS

Learn Agents by Building Small Projects



1. Weather Assistant
2. PDF Q&A Agent
3. Daily Task Planner
4. Email Drafting Agent
5. Bug Triage Agent

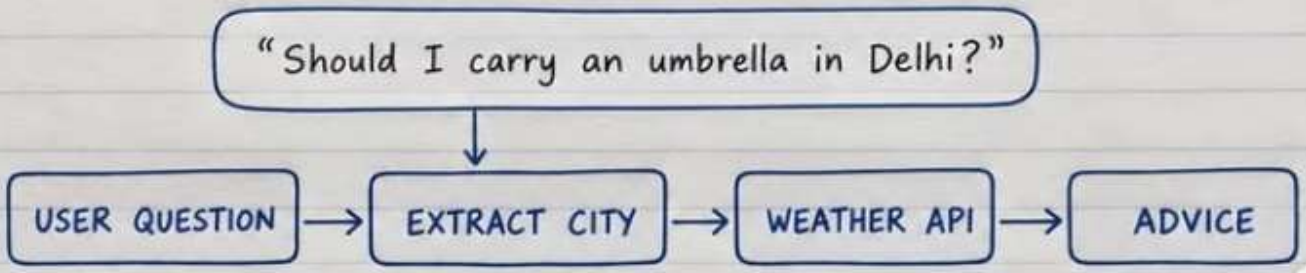
5 practical projects • Python • beginner friendly

WEATHER ASSISTANT AGENT

Project 1 • Learn Tool Calling

What It Does:

- Understands a weather question
- Extracts the city
- Calls a live weather API
- Gives useful advice



Code:

```
def weather_agent(question):
    city = extract_city(question)
    weather = get_weather(city)

    return create_advice(
        question, weather
    )
```

Tools:

- Python
- Weather API
- LLM

You Learn:

- Tool calling
- real-time data
- structured results

Key Takeaway:

The agent does not guess the weather — it calls a tool.

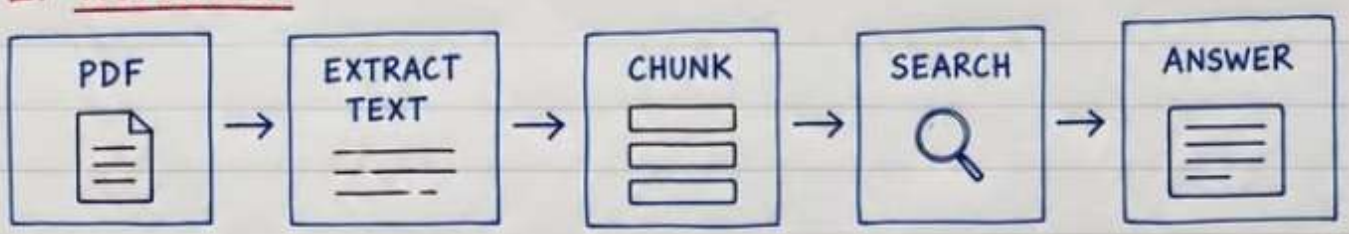
PDF QUESTION-ANSWER AGENT

Project 2 • Learn Basic RAG

1. What It Does:

- Upload a PDF or notes
- Break text into small chunks
- Find the most relevant chunks
- Answer only from the document

2. Workflow:



3. Example Question:

Which skills are mentioned in this resume?

4. Core Code (Simplified):

```
def pdf_agent(question):
    chunks = search_pdf(question)

    return ask_model(
        question=question,
        context=chunks
    )
```

5. Simple Stack:

- Python
- PDF Reader
- Embeddings
- Vector DB
- LLM

6. You Learn:

- Chunking
- retrieval
- context
- grounded answers

7. Key Takeaway:

Retrieve relevant text first; then ask the model to answer.

DAILY TASK PLANNER AGENT

Project 3 • Learn Planning

What It Does:

- Reads a list of daily tasks
- Finds priority and deadlines
- Estimates the required time
- Creates a simple schedule

Example Input:

"Finish assignment, exercise and attend meeting at 4 PM."

Workflow:



Python Code:

```
def planner_agent(tasks):
    priorities = rank_tasks(tasks)
    schedule = create_schedule(
        priorities
    )
    return schedule
```

Example Output:

```
09:00 — Assignment
13:00 — Exercise
16:00 — Meeting
```

You Learn:

Planning • structured output • prioritization

Key Takeaway:

Break one large goal into small, ordered actions.

EMAIL DRAFTING AGENT

Project 4 • Learn Human Approval

What It Does:

- Understands the email purpose
- Creates a professional draft
- Shows the draft to the user
- Sends only after approval

Example User Request:

"Write a leave request for tomorrow."

Workflow:



Python Code:

```
def email_agent(instruction):
    draft = create_email(instruction)

    if get_user_approval(draft):
        return send_email(draft)

    return draft
```

You Learn:

Human-in-the-loop • safe actions • confirmation

Safety Note:

Never send an email automatically without clear user approval.

Key Takeaway:

The agent prepares the action; the user stays in control.

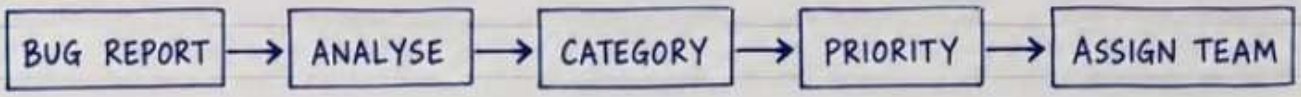
BUG TRIAGE AGENT

Project 5 • Learn Classification & Routing

What It Does:

- Reads the bug title and description
- Identifies the probable category
- Assigns priority
- Suggests the responsible team
- Produces a short summary

Workflow:



Example Input:

"Payment succeeds but order remains pending."

Python Code:

```
def bug_agent(bug):
    category = classify_bug(bug)
    priority = find_priority(bug)

    return {
        "category": category,
        "priority": priority,
        "summary": summarize(bug)
    }
```

Example Output:

```
Category: Backend
Priority: High
Assign To: Payments Team
```

You Learn:

Classification • structured JSON • intelligent routing

Key Takeaway:

A small useful agent can save repetitive manual triage work.

1. INTRODUCTION TO AI AGENTS

1. What is an AI Agent ?

- An AI Agent is an intelligent system that can perceive its environment, reason, take actions and achieve a goal autonomously (using an LLM or other AI model).
- It can interact with tools, data and other systems to complete real-world tasks.

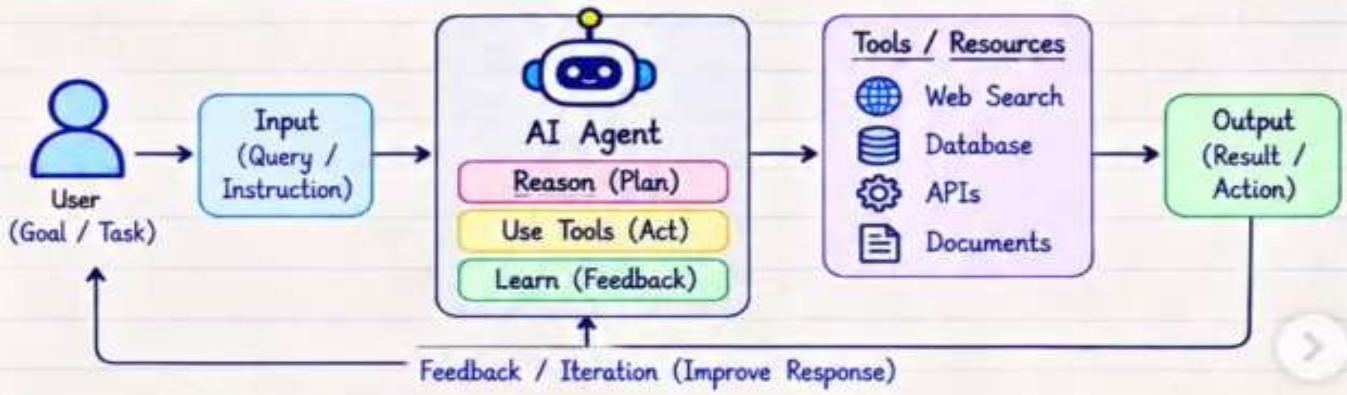
@code.withnarayan

2. Key Characteristics

- **Autonomous** : Works with minimal human input.
- **Goal-Oriented** : Performs tasks to achieve a goal.
- **Perception** : Understands input from user or environment.
- **Reasoning** : Plans and makes decisions.
- **Action** : Executes tasks using tools or APIs.
- **Adaptive** : Learns and improves from feedback.

3. How Does an AI Agent Work ?

@code.withnarayan



4. Components of an AI Agent

Component	Description
LLM / Brain	Understands, reasons and plans.
Memory	Stores past interactions, context and user preferences.
Tools	External capabilities (web search, APIs, database, code, etc.).
Planner	Breaks the goal into smaller steps.
Executor	Runs the planned steps using tools.

5. Real-World Applications

- Personal Assistant (e.g., email, calendar, tasks)
- Business Automation (e.g., report generation)
- Code Assistant (e.g., write, test and run code)
- Customer Support (e.g., intelligent chatbot)
- Research Assistant (e.g., search, summarize papers)
- E-commerce Agent (e.g., product search & compare)

6. Benefits of AI Agents

- Automates complex and multi-step tasks.
- Saves time and improves productivity.
- Works with real-time and external data.
- Can use multiple tools (APIs, web, databases).
- Personalized and context-aware responses.
- Useful in business, education and daily life.

@code.withnarayan

7. Practice Questions (MCQs)

1. What is the main characteristic of an AI Agent ?
   (a) Only generates text
   (b) Can take actions to achieve a goal
   (c) Works without any input
   (d) Only analyzes data

2. Which of the following is a component of an AI Agent ?
   (a) Compiler
   (b) Memory
   (c) Operating System
   (d) Web Browser

2. AI Agents vs Traditional AI vs Generative AI

1. Overview

- AI has evolved from rule-based systems to generative models and now to AI Agents.
- Each type has a different purpose, capability and level of autonomy.

@code.withnarayan

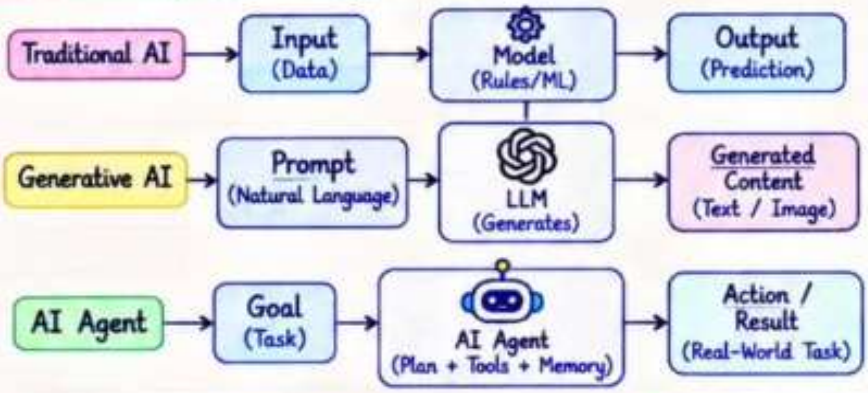
2. What are They ?

- Traditional AI : Rule-based or pre-trained systems for specific, predefined tasks.
- Generative AI : Uses large models (e.g., LLMs) to generate new content (text, image, code, etc.).
- AI Agent : An autonomous system that uses LLMs + tools + memory to achieve a goal.

3. Key Comparison

Aspect	Traditional AI	Generative AI	AI Agent
Main Goal	Solve specific, predefined problems	Generate new content (text, image, code, etc.)	Achieve a goal autonomously using tools and reasoning
Input	Structured or fixed data	Natural language (prompts)	Goal, instruction and real-time data
Output	Prediction / classification (e.g., spam or not)	Generated content (e.g., essay, image, code)	Action + result (e.g., book a ticket, send email)
Autonomy	Low (rule-based)	Medium (generates responses)	High (plans, decides and executes)
Use of Tools	No (or very limited)	Limited (mostly content generation)	Yes (APIs, database, web, etc.)
Memory	Usually not present	Limited (conversation context)	Yes (short-term & long-term)
Examples	Spam filter, loan approval	ChatGPT, DALL·E, Gemini	Travel agent, research agent, code assistant

4. Working Flow Comparison



5. When to Use ?

- Use Traditional AI when you need a simple and specific solution (e.g., classification, prediction).
- Use Generative AI when you want to create new content (e.g., text, image, code, summary).
- Use AI Agent when the task requires multiple steps, tool usage and autonomous execution (e.g., research, booking, automation).

@code.withnarayan

6. Key Takeaways

- Traditional AI → Task-specific, rule or model based.
- Generative AI → Creates new content using large models (LLMs).
- AI Agent → Autonomous, goal-driven system that can plan, use tools and act.
- AI Agents combine the power of LLMs, tools and memory to solve real-world problems.

7. Practice Questions (MCQs)

1. Which of the following is the key feature of an AI Agent ?
   (a) Only classifies data (b) Generates new content
   (c) Works autonomously to achieve a goal
   (d) Only follows fixed rules

2. Which is the correct order of evolution in AI ?
   (a) Generative AI → Traditional AI → AI Agent
   (b) Traditional AI → Generative AI → AI Agent
   (c) AI Agent → Generative AI → Traditional AI
   (d) Traditional AI → AI Agent → Generative AI

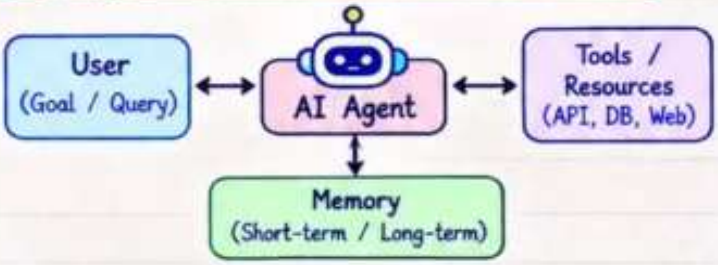
3. ARCHITECTURE OF AI AGENTS

1. What is AI Agent Architecture ?

- It is the overall structure that shows how different components of an AI Agent work together to perceive, reason, plan, use tools and take action to achieve a goal.

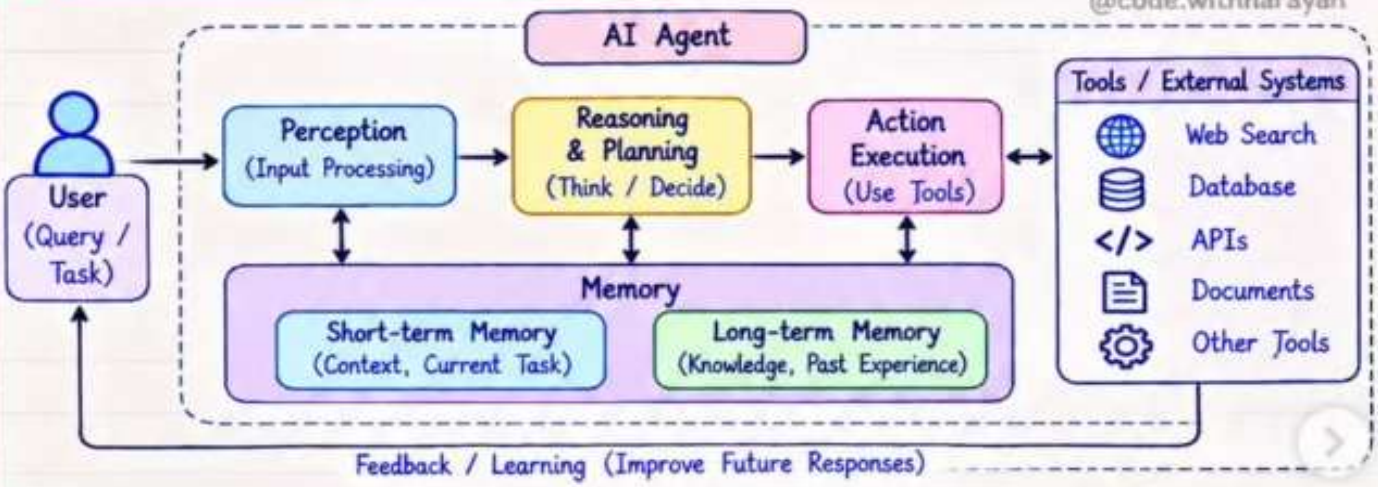
@code.withnarayan

2. High-Level Architecture (Overview)



3. Detailed Architecture of an AI Agent

@code.withnarayan



4. Core Components

Component	Description
Perception	Takes input from user or environment and understands it.
Reasoning & Planning	Analyzes the input, plans the steps and decides what to do.
Action Execution	Performs the planned actions using tools, APIs or other systems.
Memory	Stores context, past interactions and useful knowledge (short-term & long-term).
Tools	External capabilities like web search, database, file system, code execution, etc.
Feedback / Learning	Learns from results and improves future responses.

5. Architecture Flow (Step by Step)

1. Receive a goal or query from the user.
2. Perceive and understand the input.
3. Reason and create a plan (multi-step if needed).
4. Use tools (e.g., web search, API, database).
5. Take action and generate the final output.
6. Store important information in memory.
7. Use feedback to improve future responses.

6. Key Takeaways

- AI Agent = Perception + Reasoning + Action + Memory + Tools + Learning.
- Memory (short-term & long-term) makes the agent context-aware.
- Tools extend the agent's real-world capabilities.
- It works in a loop with feedback for continuous improvement.

7. Practice Questions (MCQs)

1. Which of the following is NOT a core component of an AI Agent ?
   (a) Perception (b) Reasoning & Planning
   (c) Compiler (d) Memory

2. What is the main role of Memory in an AI Agent ?
   (a) To compile the code (b) To store past interactions
   (c) To generate images (d) To translate language

@code.withnarayan

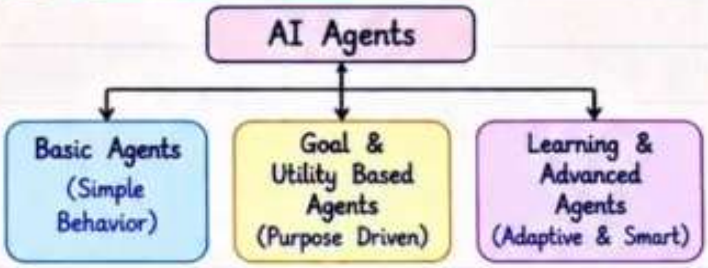
4. TYPES OF AI AGENTS

1. What are Types of AI Agents ?

- AI Agents can be classified into different types based on their capabilities, decision-making style, memory, and learning ability.
- Each type is suitable for different kinds of problems and applications.

@code.withnarayan

2. Classification Overview



3. Types of AI Agents (with Examples)

Type of Agent	Description	Example
1. Simple Reflex Agent	Takes action based on the current input using fixed rules.	Thermostat, Spam filter
2. Model-Based Reflex Agent	Uses an internal model of the environment to make better decisions.	Robot vacuum, Game agent
3. Goal-Based Agent	Chooses actions to achieve a specific goal.	Route planner (e.g., Google Maps)
4. Utility-Based Agent	Selects the best action by maximizing a utility (e.g., reward, performance, cost).	Recommendation system (e.g., Netflix)
5. Learning Agent	Improves its performance from past experience (learns from data).	Chatbot that gets better over time
6. Multi-Agent System	A group of agents that work together to achieve a common goal.	AI team for task automation (e.g., CrewAI, AutoGen)

4. Working of Each Type (Intuition)



@code.withnarayan

5. Key Differences

Aspect	Simple Reflex	Goal-Based	Utility-Based	Learning
Uses Memory	✗	✓	✓	✓
Has a Goal	✗	✓	✓	✓
Optimizes (Utility)	✗	✗	✓	✓
Learns from Experience	✗	✗	✗	✓

6. Real-World Examples

- Chatbot → Learning Agent
- Self-Driving Car → Utility-Based Agent
- Product Recommender → Goal/Utility Agent
- AI Team (Multi-Agent) → Multi-Agent System
- Game Playing AI → Model-Based Agent

7. Key Takeaways

- Different types of AI Agents are designed for different levels of intelligence and tasks.
- Simple → Rule-based, Goal-based → Purpose-driven, Utility-based → Best outcome, Learning → Improves over time.
- Multi-Agent systems enable collaboration among multiple agents.

@code.withnarayan

8. Practice Questions (MCQs)

1. Which type of agent learns from past experience ?
   (a) Simple Reflex Agent (b) Goal-Based Agent
   (c) Utility-Based Agent (d) Learning Agent
2. Which agent selects the best action by maximizing a utility function ?
   (a) Reflex Agent (b) Model-Based Agent
   (c) Utility-Based Agent (d) Multi-Agent System

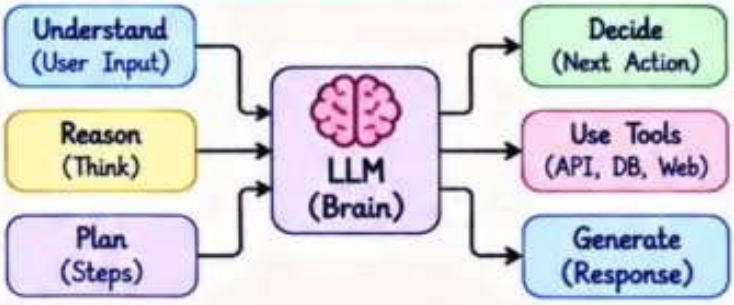
5. LLMs as the Brain of AI Agents

1. Why LLMs are the Brain ?

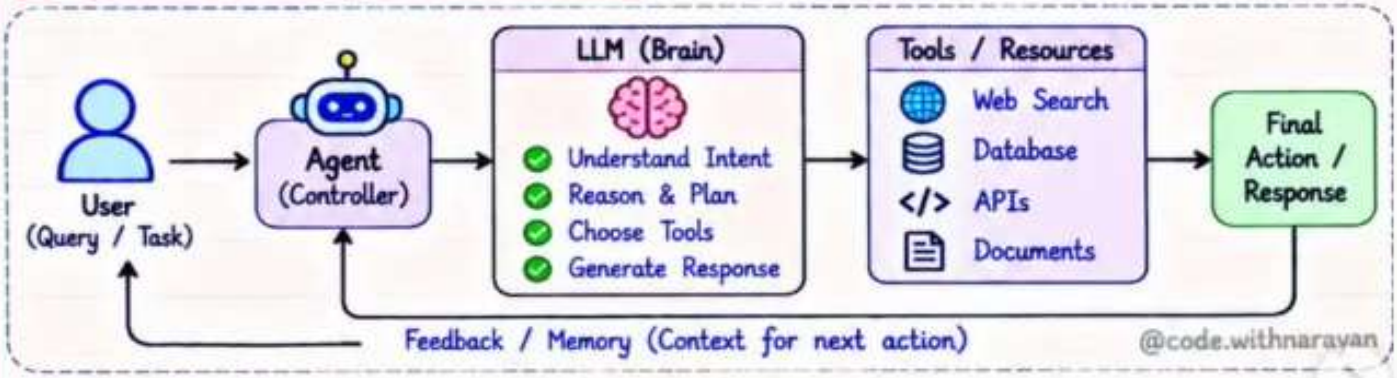
- LLMs (Large Language Models) act as the **brain** of an AI Agent because they **understand**, **reason**, **plan**, **decide** and **generate actions** in natural language.
- They provide the core intelligence that allows the agent to think and solve complex, multi-step tasks.

@code.withnarayan

2. Role of LLM in an AI Agent



3. LLM-Based AI Agent Architecture



@code.withnarayan

4. Key Capabilities Enabled by LLMs

- **Language Understanding** : Understands user goals and context.
- **Reasoning** : Breaks down complex tasks and finds a solution.
- **Planning** : Creates a step-by-step action plan.
- **Tool Use** : Decides which tool (API, DB, Web) to use and how.
- **Memory Handling** : Uses past context to give better and consistent results.
- **Natural Interaction** : Communicates in easy and human-like language.

5. Example : LLM as Agent Brain

User Query

"Book a train ticket to Delhi for tomorrow."

LLM (Brain) – Agent Thinking

1. **Understand** : Book train ticket.
2. **Plan** : Check availability → Select train → Book ticket.
3. **Use Tools** : Call Train API (IRCTC or similar).
4. **Action** : Book the ticket.
5. **Response** : "Your ticket is booked successfully."

6. Popular LLMs Used in AI Agents

LLM	Provider	Key Strength
GPT-4 / GPT-4o	OpenAI	Strong reasoning, tool use, multimodal.
Claude 3	Anthropic	Long context, safety, reliable.
Gemini 1.5	Google	Multimodal, large context window.
Llama 3	Meta	Open-source, cost-effective.
Mistral	Mistral AI	Fast, lightweight, good performance.

@code.withnarayan

7. Key Takeaways

- LLM is the core brain of an AI Agent.
- It helps the agent to understand, reason, plan and take actions.
- It works with tools, memory and external data.
- Popular LLMs: GPT-4, Claude, Gemini, Llama, Mistral.

8. Practice Questions (MCQs)

1. Why is an LLM considered the brain of an AI Agent ?
2. Which of the following is a popular LLM used in agents ?
   (a) GPT-4 (b) MySQL (c) Docker (d) Kafka

6. TOOLS & FUNCTION CALLING

1. What are Tools in AI Agents ?

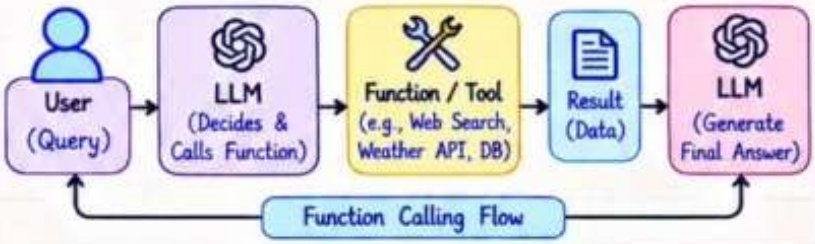
- Tools are external capabilities that an AI Agent can use to interact with the real world (e.g., search the web, access a database, run code, send an email, etc.).
- They extend the ability of an LLM to perform real and practical tasks beyond just generating text.

@code.withnarayan

2. What is Function Calling ?

- Function Calling is a feature that allows an LLM to call a predefined function (tool) with structured arguments (e.g., JSON) when needed.
- The model decides which function to call, passes the required input and uses the returned result to generate the final response.

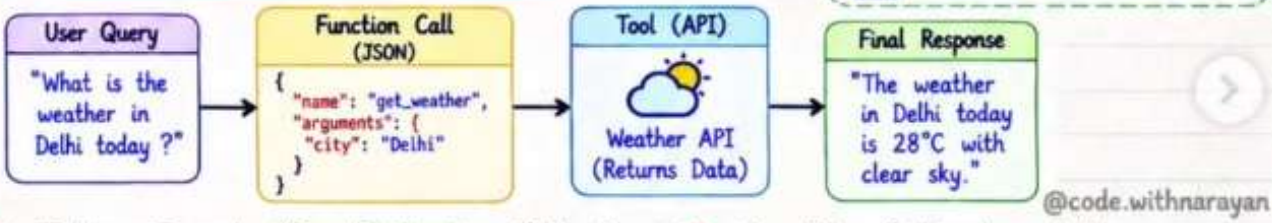
3. How Tools & Function Calling Work ?



4. Common Tools for AI Agents

- Web Search (e.g., Google, Bing)
- Database (e.g., PostgreSQL)
- Code Execution (e.g., Python)
- File Operations (read, write, upload)
- Email / Notification (send email)
- Productivity (calendar, to-do, etc.)

5. Function Calling Example (Concept)



@code.withnarayan

6. Python Example (OpenAI Function Calling)

Function Calling (Python)

```python
from openai import OpenAI
client = OpenAI()
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather of a city",
        "parameters": {
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"]
        }
    }
}]
response = client.chat.completions.create(
  model="gpt-4o",
  messages=[{"role": "user", "content": "Weather in Delhi?"}],
  tools=tools
)
```

@code.withnarayan

7. Benefits of Function Calling

- Lets LLMs perform real-world actions.
- Provides structured and reliable results.
- Connects with APIs, databases and custom logic.
- Makes AI Agents more useful and practical.
- Improves accuracy and reduces hallucination.

8. Key Takeaways

- Tools extend the skills of an AI Agent.
- Function Calling lets the LLM use these tools.
- It works using structured inputs (e.g., JSON).
- Common tools: web search, database, code, email, etc.

9. Practice Questions (MCQs)

1. What is the main purpose of Function Calling in an AI Agent ?
   (a) To generate images
   (b) To call external tools or functions
   (c) To store user data
   (d) To train a new model

2. Which of the following is an example of a tool used by an AI Agent ?
   (a) Web Search API
   (b) PostgreSQL Database
   (c) Python Code Execution
   (d) All of the above

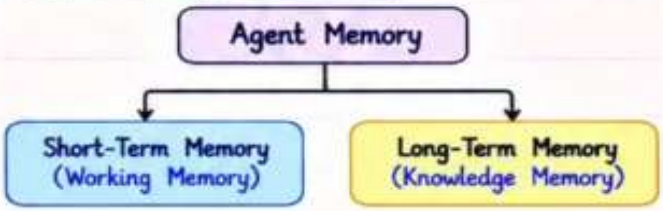
@code.withnarayan

7. AGENT MEMORY — SHORT-TERM & LONG-TERM MEMORY

1. What is Agent Memory ?

- Agent Memory is the mechanism that helps an AI Agent to **remember** information, **maintain context** and **use past experience** for better decisions.
- It gives the agent continuity and personalization across interactions.

2. Types of Agent Memory



@code.withnarayan

3. Short-Term Memory (STM)

- Stores temporary information during a single conversation or task.
- Helps to maintain the current context (e.g., previous messages, user intent, intermediate results).
- It is limited and gets cleared when the session ends.

Example (STM)

User : "Book a train ticket."
Agent : "From which city ?"
User : "Delhi to Mumbai."

The agent remembers "Book a ticket" and uses it to continue the conversation.

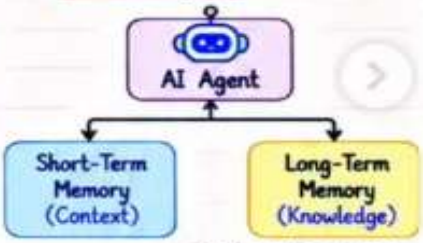
4. Long-Term Memory (LTM)

- Stores important and persistent information for a longer period.
- Includes user preferences, past interactions, domain knowledge, and learned experience.
- It helps in personalization and improves the quality of future responses.

5. Comparison : Short-Term vs Long-Term Memory

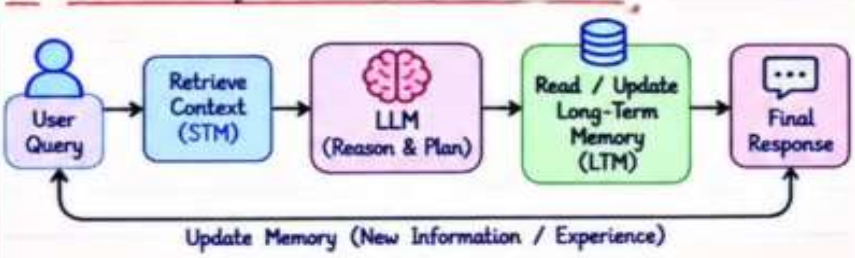
Aspect	Short-Term Memory (STM)	Long-Term Memory (LTM)
Purpose	Maintain current context	Store long-lasting knowledge
Duration	Temporary (session/task level)	Persistent (across sessions)
Capacity	Limited (few messages / tokens)	Large (documents, history, embeddings)
Examples	Recent chat messages, current inputs	User profile, past tasks, domain data
Clearing	Cleared after session ends	Remains until updated or deleted

6. Memory Architecture in AI Agent



@code.withnarayan

7. How Memory Works in an AI Agent ?



8. Real-World Examples

- **Chatbot** : Remembers user's past queries and preferences.
- **Travel Agent** : Saves past bookings and favorite routes.
- **Study Assistant** : Remembers the topics you are learning.
- **Customer Support Agent** : Stores user history and previous issues.

9. Key Takeaways

- Agent Memory makes the agent context-aware and more useful.
- Short-Term Memory (STM) → current, temporary context.
- Long-Term Memory (LTM) → persistent, long-lasting knowledge.
- Both together enable personalization, continuity and better decision making.

@code.withnarayan

10. Practice Questions (MCQs)

1. Which type of memory maintains the current conversation context ?
   (a) Long-Term Memory (b) Short-Term Memory
   (c) External Memory (d) None of the above
2. Which memory stores user preferences and past history ?
   (a) Short-Term Memory (b) Long-Term Memory
   (c) Working Memory (d) Cache Memory

8. PLANNING, REASONING & DECISION MAKING

1. Overview

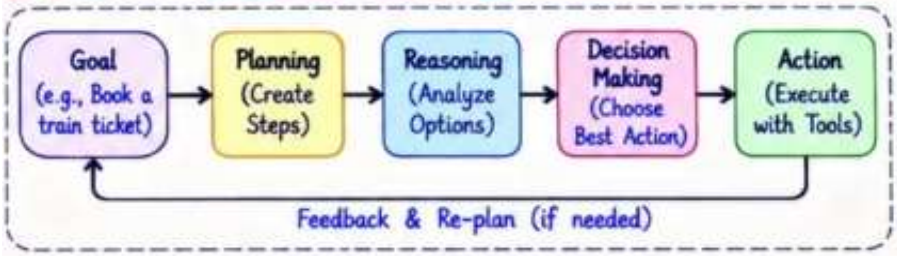
- Planning, Reasoning and Decision Making are the core cognitive abilities of an AI Agent that help it to achieve a goal autonomously.
- These work together to analyze a problem, create a plan, choose the best action and execute it.

@code.withnarayan

2. Key Concepts

Planning	Breaks a goal into smaller, step-by-step actions to achieve it.
Reasoning	Uses logic and knowledge to analyze, infer and solve a problem.
Decision Making	Selects the best action from available choices based on the current situation, goal and constraints.

3. How They Work Together ?



4. Planning in AI Agents

- Converts a high-level goal into a sequence of actionable steps.
- Common techniques :
  1. Chain-of-Thought (CoT)
  2. Task Decomposition
  3. Plan-and-Execute
  4. ReAct (Reason + Act)

5. Reasoning in AI Agents

- Helps the agent to think logically, understand context and draw useful conclusions.
- Types of Reasoning :
  1. Deductive Reasoning : General → Specific
  2. Inductive Reasoning : Specific → General
  3. Abductive Reasoning : Best possible explanation

6. Decision Making in AI Agents

- Chooses the most appropriate action using the plan, available tools, and current situation.
- Common Approaches :

@code.withnarayan

Approach	Idea	Example
Rule-Based	Follow predefined rules	If price < 1000, book ticket
Utility-Based	Choose action with highest benefit	Select cheapest & fastest option
Probabilistic	Decide under uncertainty	Choose option with highest probability of success
LLM-Based	Use LLM reasoning to decide	LLM compares options and picks the best one

7. Example : End-to-End Flow

Goal : "Book a train ticket from Delhi to Mumbai"

Planning	Break into steps: search train, compare options, book ticket.
Reasoning	Check availability, timing and price.
Decision Making	Choose the best train (e.g., fastest & cheapest).
Action	Call Train API and book the ticket.

Output : "Ticket booked successfully (PNR: 123456)"

8. Real-World Applications

- Travel Agent – Plan trip, choose best route and book tickets.
- Shopping Agent – Compare products and select the best deal.
- Coding Agent – Plan, reason and generate code solutions.
- Research Agent – Plan research, fetch data and summarize results.

9. Key Takeaways

- Planning creates a step-by-step roadmap to the goal.
- Reasoning provides logical thinking and problem solving.
- Decision Making selects the best action from available options.
- Together, they make the AI Agent autonomous, intelligent and goal-oriented.

@code.withnarayan

10. Practice Questions (MCQs)

1. Which component creates a step-by-step plan to achieve a goal ?
   (a) Reasoning (b) Planning
   (c) Decision Making (d) Memory
2. Which reasoning type finds the best possible explanation ?
   (a) Deductive (b) Inductive
   (c) Abductive (d) Probabilistic

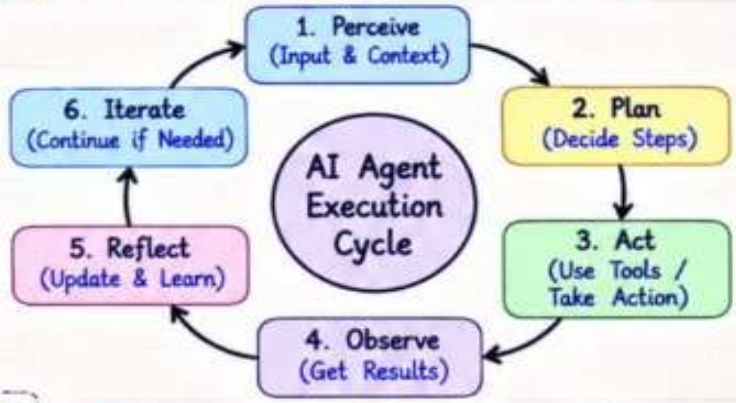
9. AGENT WORKFLOW & EXECUTION CYCLE

1. What is Agent Workflow ?

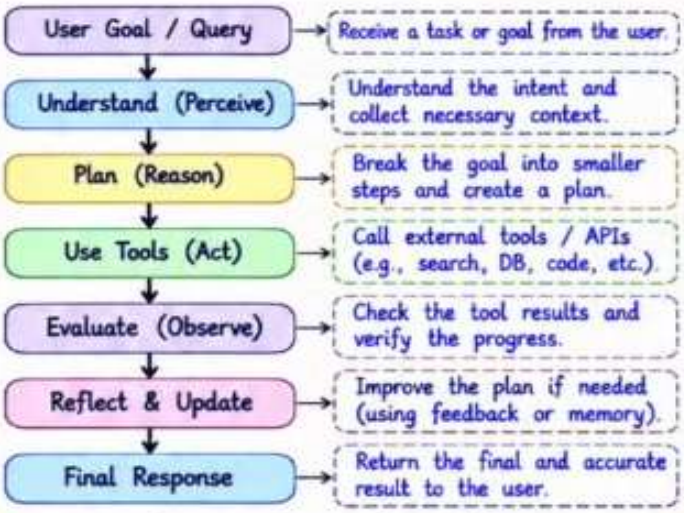
- Agent Workflow is the step-by-step process that an AI Agent follows to understand a goal, plan, use tools, take action and deliver the final result.
- It defines how different components (LLM, tools, memory, planner) work together in a continuous cycle.

@code.withnarayan

2. Agent Execution Cycle (Overview)



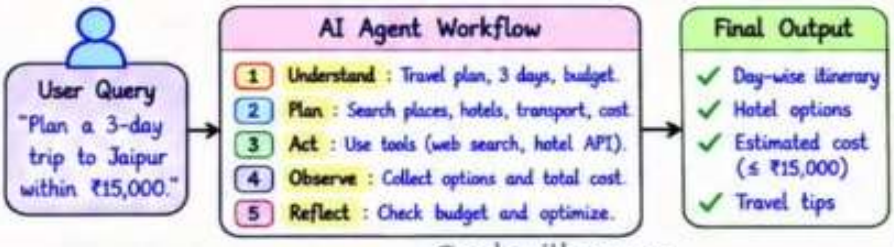
3. Detailed Agent Workflow



4. Key Stages of Execution Cycle

Stage	Purpose	Example
Perceive	Take input and understand the task.	"Find best laptop under ₹50,000."
Plan	Create a step-by-step plan.	Search → Compare → Shortlist → Recommend
Act	Use tools and take action.	Call web search / price API.
Observe	Analyze the results.	Get price, rating and features.
Reflect	Check if goal is achieved and improve if needed.	If not, refine search and try again.
Iterate	Repeat the cycle until the goal is completed.	Continue until best recommendation is found.

5. Example : End-to-End Workflow



@code.withnarayan

6. Common Workflow Patterns

1. **ReAct Workflow** : Reason + Act + Observe (iterative).
2. **Plan-and-Execute** : Plan first, then execute all steps.
3. **Self-Refine** : Generate → Critique → Improve.
4. **Multi-Agent Workflow** : Multiple agents work together (e.g., planner, researcher, writer).

7. Benefits of a Well-Defined Workflow

- Makes the agent more reliable and consistent.
- Helps in solving complex and multi-step tasks.
- Enables the use of external tools and real-time data.
- Supports iteration, error handling and self-improvement.
- Improves accuracy and the quality of the final result.

@code.withnarayan

8. Practice Questions (MCQs)

1. Which is the correct order in an AI Agent execution cycle ?
   (a) Plan → Act → Observe → Reflect
   (b) Perceive → Plan → Act → Observe → Reflect → Iterate
   (c) Act → Plan → Reflect → Observe
   (d) Observe → Act → Plan → Iterate

2. Which workflow pattern uses reasoning and tool execution in an iterative manner ?
   (a) Plan-and-Execute (b) ReAct
   (c) Self-Refine (d) Multi-Agent

@code.withnarayan

9. Key Takeaways

- Agent Workflow defines the complete flow from input to final result.
- The execution cycle is: Perceive → Plan → Act → Observe → Reflect → Iterate.
- Different workflow patterns (ReAct, Plan-and-Execute, Self-Refine) are used based on the task complexity.

10. AI AGENT FRAMEWORKS – LangChain, LangGraph, CrewAI & AutoGen

1. What are AI Agent Frameworks ?

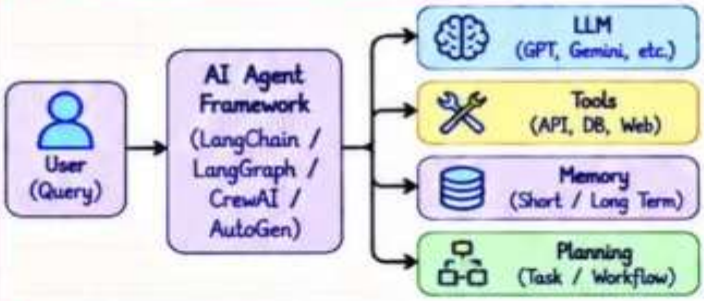
- AI Agent Frameworks are ready-to-use libraries and tools that help to build, run and manage AI Agents easily.
- They provide components like LLM integration, tools, memory, planning and multi-agent collaboration.

@code.withnarayan

2. Overview of Popular Frameworks

LangChain	• Popular framework to build LLM applications and AI Agents with tools, memory and more.
LangGraph	• A graph-based extension of LangChain for building stateful, multi-step and complex agents.
CrewAI	• A lightweight framework to create and manage multiple AI agents (a team) for a common goal.
AutoGen	• A multi-agent framework by Microsoft for building collaborative and conversational agents.

3. Architecture (High-Level View)



4. Key Features of Each Framework

Framework	Key Features
LangChain	• Build LLM apps and simple to complex agents. • Tools, memory, chains and integrations.
LangGraph	• Graph-based, stateful workflows. • Best for long-running and complex agents.
CrewAI	• Multi-agent (team) collaboration. • Role-based agents, easy to set up.
AutoGen	• Multi-agent conversation and collaboration. • Supports code execution, tools and human-in-loop.

5. Comparison : LangChain vs LangGraph vs CrewAI vs AutoGen

Feature	LangChain	LangGraph	CrewAI	AutoGen
Main Focus	Build LLM apps & agents	Stateful agent workflows	Multi-agent team workflow	Multi-agent collaboration
Workflow Style	Chain / Modular	Graph (Nodes + Edges)	Role-based	Conversational
Multi-Agent	Limited (with tools)	Yes (via graph)	Yes (core)	Yes (core)
Best For	General use & quick agents	Complex, stateful applications	Task delegation & team agents	Research, coding & collaborative tasks

6. When to Use Which ?

- **LangChain** : When you want to quickly build an LLM application or a single agent.
- **LangGraph** : When you need complex, stateful and multi-step agent workflows.
- **CrewAI** : When you want a team of agents working together.
- **AutoGen** : When you need collaborative, multi-agent conversations (e.g., research, coding).

7. Simple Code Example (LangChain)

LangChain – Simple Agent

```python
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, Tool

# Define a tool
def get_time():
    return "Current time: 10:30 AM"

tools = [Tool(name="GetTime", func=get_time,
              description="Get current time")]

llm = ChatOpenAI(model="gpt-4o")
agent = initialize_agent(tools, llm, agent="zero-shot-react")

agent.run("What is the current time?")
```

@code.withnarayan

8. Real-World Use Cases

@code.withnarayan

- **Research Agent** – Collects, summarizes and analyzes information from the web.
- **Business Agent** – Automates reports, data analysis and insights.
- **Coding Agent** – Writes, runs and debugs code (e.g., AutoGen / CrewAI).
- **Multi-Agent Team** – A team of agents (researcher, writer, reviewer) to create a complete report.

9. Key Takeaways

- AI Agent frameworks make it easy to build and manage agents.
- LangChain → general & flexible, LangGraph → stateful workflows, CrewAI → multi-agent teams, AutoGen → collaborative agents.
- Choose the framework based on task complexity and workflow needs.

Practice Questions (MCQs)

1. Which framework is best for graph-based, stateful agent workflows ?
   (a) LangChain (b) LangGraph
   (c) CrewAI (d) AutoGen

2. Which framework is primarily designed for multi-agent (AI team) collaboration ?
   (a) LangChain (b) LangGraph
   (c) CrewAI (d) AutoGen

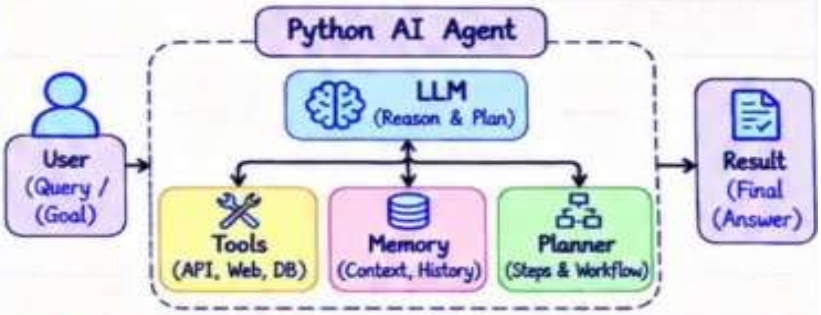
11. BUILDING AI AGENTS USING PYTHON

1. What is an AI Agent in Python?

- An AI Agent is a Python program that uses an LLM, tools and memory to perceive a goal, plan actions and execute tasks automatically.
- We can build AI Agents in Python using frameworks like LangChain, LangGraph, CrewAI or AutoGen.

@code.withnarayan

2. High-Level Architecture



3. Steps to Build an AI Agent (Python)

1. **Setup Environment** → Install Python and required libraries (e.g., LangChain, OpenAI).
2. **Configure LLM** → Use an LLM (e.g., GPT) with API key.
3. **Add Tools** → Integrate external tools (e.g., web search, calculator, database).
4. **Add Memory** → Keep conversation history (e.g., ConversationBufferMemory).
5. **Create Agent** → Define the agent with tools and a reasoning style (e.g., ReAct).
6. **Run & Test** → Give a goal/query and verify the agent's response.

4. Python Code Example (LangChain)

Build a Simple AI Agent

```python
from langchain_openai import ChatOpenAI
from langchain.agents import AgentExecutor, create_react_agent
from langchain.tools import Tool

# 1. Define a tool
def get_time():
    from datetime import datetime
    return datetime.now().strftime("%I:%M %p")

# 2. Create tool list
tools = [Tool(name="GetTime", func=get_time,
              description="Get current time.")]

# 3. Initialize LLM
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 4. Create agent (ReAct style)
agent = create_react_agent(llm, tools)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# 5. Run the agent
response = agent_executor.invoke({"input": "What is the current time?"})
print(response["output"])
```

@code.withnarayan

5. Code Explanation

Code Part	Purpose
Tool (get_time)	A custom tool that returns current time.
tools = [...]	List of tools that the agent can use.
ChatOpenAI(...)	Connects to an LLM (e.g., GPT-4o-mini).
create_react_agent()	Creates a ReAct-style agent (Reason + Act).
AgentExecutor(...)	Runs the agent and manages the workflow.
invoke({"input": ...})	Sends the user query and gets the result.

6. Output (Example)

User Input :
What is the current time?

Agent Output :
The current time is 10:30 AM.

7. Real-World Use Cases

- **Research Agent** – Search, read and summarize information.
- **Productivity Agent** – Manage tasks, emails and schedules.
- **Coding Agent** – Write, run and debug code.
- **Data Agent** – Fetch, analyze and create reports.

@code.withnarayan

8. Key Takeaways

- Python is a popular and flexible language to build AI Agents.
- Use an LLM for reasoning, tools for real-world actions and memory for context.
- Frameworks like LangChain, LangGraph, CrewAI and AutoGen make agent development easier.
- Start with a simple agent and then add more tools, memory and workflows.

@code.withnarayan

9. Practice Questions (MCQs)

1. Which Python framework is widely used to build AI Agents ?
   (a) NumPy (b) LangChain
   (c) Pandas (d) Matplotlib

2. In the given code, what is the role of AgentExecutor ?
   (a) To create an LLM model
   (b) To run the agent and manage its workflow
   (c) To store conversation history
   (d) To define a custom tool

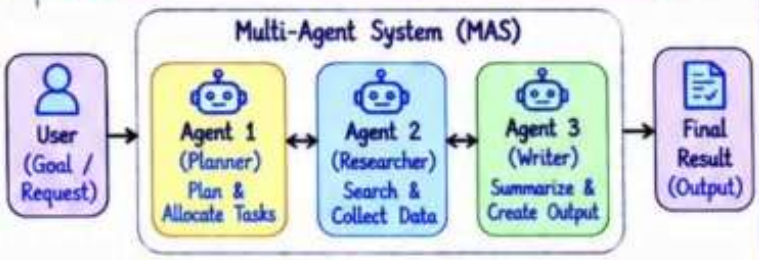
12. MULTI-AGENT SYSTEMS & COLLABORATION

1. What is a Multi-Agent System (MAS) ?

- A Multi-Agent System (MAS) is a system of multiple AI agents that work together to achieve a common goal.
- Each agent has a specific role, capability and can communicate, coordinate and collaborate with other agents.

@code.withnarayan

2. Architecture of a Multi-Agent System



3. How Multi-Agent Collaboration Works ?

1. **Goal Definition** → User gives a high-level goal (e.g., "Create a research report").
2. **Task Decomposition** → The planner agent breaks the goal into smaller tasks.
3. **Task Execution** → Specialized agents work on their assigned tasks in parallel.
4. **Collaboration** → Agents share results, discuss and refine the output.
5. **Final Output** → The system combines results and returns the final answer.

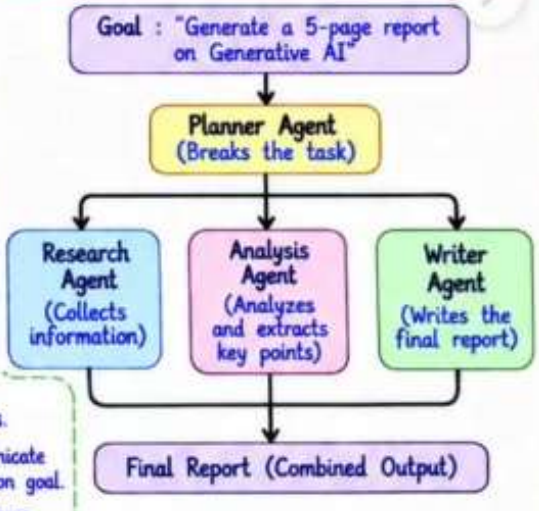
4. Key Collaboration Patterns

Pattern	Description
Hierarchical	• One main (manager) agent assigns tasks to sub-agents.
Sequential	• Agents work in a fixed order (one after another).
Parallel	• Multiple agents work at the same time on different tasks.
Peer-to-Peer	• Agents collaborate as equals and share information directly.
Debate / Reflect	• Agents discuss, critique and improve each other's results.

5. MAS vs Single Agent (Comparison)

Aspect	Single Agent	Multi-Agent System (MAS)
Agents	One agent	Multiple agents
Task Handling	Handles all tasks	Tasks are divided among agents
Complexity	Simple tasks	Complex, multi-step tasks
Efficiency	Limited (depends on one agent)	Higher (parallel & specialized work)
Use Case	Q&A, simple automation	Research, report generation, project management
Example	Single chatbot	Team of agents (Planner + Researcher + Writer)

6. Example : Research Report using MAS



@code.withnarayan

7. Real-World Use Cases

- **Research Assistant** – Multiple agents for search, analysis and report writing.
- **Software Development** – Agents for planning, coding, testing and documentation.
- **Business Analysis** – Agents to collect data, analyze and generate insights.
- **Customer Support** – Agents for query classification, response and escalation.

8. Key Takeaways

- Multi-Agent Systems use multiple specialized agents.
- Agents collaborate, communicate and work towards a common goal.
- Different collaboration patterns (e.g., hierarchical, parallel, sequential) can be used.
- MAS is useful for complex, real-world and multi-step tasks.

9. Practice Questions (MCQs)

1 What is a Multi-Agent System (MAS) ?

(a) A system with a single AI agent
(b) A system of multiple agents working together
(c) A chatbot with memory
(d) A model for only data analysis

2 Which collaboration pattern allows multiple agents to work at the same time ?

(a) Sequential (b) Hierarchical
(c) Parallel (d) Debate

13. RAG-BASED AI AGENTS

1. What is a RAG-Based AI Agent?

- A RAG (Retrieval-Augmented Generation) based AI Agent combines a LLM with an external knowledge source (e.g., documents, web, database) to give accurate, up-to-date and context-aware answers.
- It first retrieves relevant information and then uses the LLM to generate a final, natural language response.

@code.withnarayan

2. RAG Architecture (AI Agent with RAG)



3. Working of a RAG-Based Agent



4. Components of a RAG-Based Agent

Component	Role / Description
LLM	• Generates the final natural language answer using the retrieved context.
Retriever	• Finds the most relevant information from the knowledge source.
Embedding Model	• Converts text into vectors for semantic search (e.g., OpenAI, HuggingFace).
Vector Database	• Stores document embeddings for fast and accurate retrieval (e.g., Chroma, FAISS, Pinecone).
Knowledge Source	• External data such as PDFs, docs, web pages, Notion, database, etc.

@code.withnarayan

5. RAG vs Normal LLM (Comparison)

Aspect	Normal LLM	RAG-Based AI Agent
Knowledge	Limited to training data	Uses external, updated data
Answer Accuracy	May be outdated or wrong	More accurate and reliable
Domain Specific	Not always good	Very good (e.g., company docs)
Hallucination	Higher (can make up facts)	Lower (uses real retrieved data)
Use Case	General questions (chat)	Domain-specific, real-time, enterprise and knowledge-based tasks

6. Simple Example (Code – LangChain)

RAG Agent – Python (LangChain)

```python
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import Chroma
from langchain_openai import ChatOpenAI
from langchain_chains import RetrievalQA

# Load and index a PDF
docs = PyPDFLoader("notes.pdf").load()
vector_db = Chroma.from_documents(docs)
# Create RAG Agent (QA Chain)
llm = ChatOpenAI(model="gpt-4o-mini")
rag_agent = RetrievalQA.from_chain_type(
    llm=llm, retriever=vector_db.as_retriever()
)
# Ask a question
print(rag_agent.invoke({"query": "What is RAG?"})
```

@code.withnarayan

7. Real-World Use Cases

- **Document Q&A** – Ask questions on PDFs, research papers or company docs.
- **Customer Support Agent** – Answer user queries using product documentation.
- **Learning Assistant** – Explain concepts using course material or notes.
- **Enterprise Search** – Get answers from internal company knowledge base (Wiki, Notion, etc.).

8. Key Takeaways

- RAG combines retrieval of relevant data with LLM generation.
- It reduces hallucination and improves accuracy.
- Uses components like retriever, embeddings, vector DB and LLM.
- Useful for domain-specific, real-time and knowledge-grounded AI Agents.

9. Practice Questions (MCQs)

1. What does RAG stand for ?
   (a) Retrieval-Augmented Generation
   (b) Real-time AI Generation
   (c) Retrieval and Generation Model
   (d) Reinforced Agent Generation

2. Which component is used to store document embeddings in a RAG system ?
   (a) LLM (b) Vector Database
   (c) Planner (d) Prompt Template

14. AGENT EVALUATION, SECURITY & RESPONSIBLE AI

1. Why Evaluate, Secure & Use Responsible AI ?

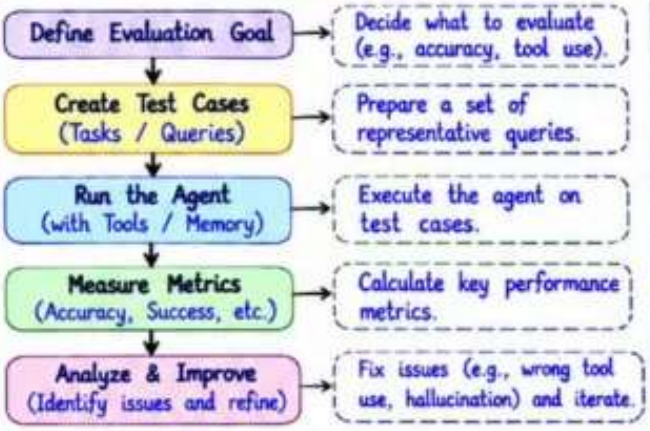
- AI Agents are powerful and can take actions, use tools and access data.
- Therefore, we must ① evaluate their performance, ② ensure security and ③ follow responsible AI practices for safe, fair and trustworthy use.

@code.withnarayan

2. Agent Evaluation – Key Metrics

Metric	Meaning (What it Measures)
Accuracy	Correctness of the agent's final answer.
Task Success Rate	% of tasks completed successfully.
Faithfulness	Whether the answer is grounded in the given data.
Tool Use Accuracy	Correct selection and execution of tools.
Latency (Time)	Time taken to complete a task.
Cost (Tokens)	Number of tokens and overall execution cost.

3. Evaluation Process (Workflow)



4. Security in AI Agents

- **Data Protection** : Keep sensitive data (PII, passwords, API keys) secure.
- **Access Control** : Give only required permissions to tools and data.
- **Secure Tool Use** : Validate inputs and prevent unauthorized actions.
- **Secrets Management** : Store API keys in environment variables / secret manager.
- **Risk Prevention** : Avoid prompt injection, data leakage and unsafe operations.

@code.withnarayan

5. Responsible AI for Agents

- **Fairness** : Avoid biased or discriminatory outputs.
- **Transparency** : Be clear about what the agent can do and its limitations.
- **Privacy** : Respect user data and follow data protection rules.
- **Safety** : Prevent harmful, illegal or misleading outputs.
- **Accountability** : Track actions (logs) and take responsibility for outcomes.

6. Common Risks & Mitigations

Risk / Issue	Mitigation (How to Handle)
Hallucination (Wrong Answer)	Use RAG, source verification and evaluation.
Prompt Injection (Manipulation)	Validate inputs, use filters and guardrails.
Data Leakage	Avoid sharing sensitive data, use access control.
Tool Misuse	Restrict tool permissions and validate actions.
Bias in Output	Use diverse data, follow responsible AI guidelines.
High Cost	Optimize prompts, set token limits and use caching.

7. Best Practices (Checklist)

- ✅ Define clear evaluation metrics and test regularly.
- ✅ Use guardrails (input/output validation).
- ✅ Follow security best practices (access control, secrets).
- ✅ Design for responsible AI (fair, safe, transparent).
- ✅ Monitor in production (logs, errors, cost, performance).

8. Key Takeaways

- ✅ Evaluate agents using clear and measurable metrics.
- ✅ Security is essential to prevent misuse and data risks.
- ✅ Responsible AI ensures fair, safe and trustworthy agents.
- ✅ Use guardrails, monitoring and continuous improvement.

9. Practice Questions (MCQs)

1 Which metric measures the success of an AI Agent in completing a task ?

(a) Latency (b) Task Success Rate
(c) Bias Score (d) Token Count

2 Which of the following is a responsible AI practice ?

(a) Using unverified data (b) Ignoring user privacy
(c) Ensuring fairness and transparency
(d) Allowing unrestricted tool access

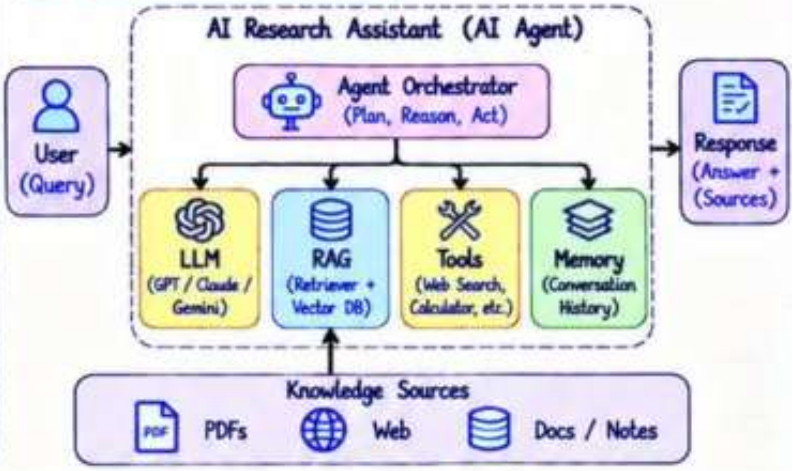
15. REAL-WORLD AI AGENT PROJECT & INTERVIEW REVISION

1. Project Overview – AI Research Assistant

- **Goal** : Build an AI Agent that can search, read, summarize and answer questions from real-world documents (PDFs / Web / Articles).
- **Features** :
  - ✓ Natural language Q&A
  - ✓ RAG (retrieve from your documents)
  - ✓ Use tools (web search, calculator)
  - ✓ Keep conversation memory
  - ✓ Source-cited answers

@code.withnarayan

2. System Architecture



3. Tech Stack

Layer	Technology
Language	Python
Framework	LangChain (or LangGraph)
LLM	OpenAI / Gemini / Claude
Vector DB	Chroma / FAISS / Pinecone
Tools	Tavily (Web Search), Calculator
App (Optional)	Streamlit / FastAPI
Data	PDFs, Docs, Web Articles

4. Project Workflow (Step-by-Step)

1. **Setup** → Install Python and required libraries (langchain, openai, chromadb, streamlit, etc.).
2. **Load Data** → Ingest documents (PDFs) and create vector store (embeddings).
3. **Define Tools** → Add tools like web search and calculator.
4. **Build Agent** → Create an agent with LLM + RAG + Tools + Memory.
5. **Build UI** → (Optional) Create a simple UI using Streamlit.
6. **Run & Test** → Ask real questions and get grounded answers with sources.

5. Minimal Code (LangChain)

AI Research Assistant (Core Code)

```python
from langchain_openai import ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain.chains import RetrievalQA

# Load documents
loader = PyPDFLoader("notes.pdf")
docs = loader.load()
# Create vector store
vector_db = Chroma.from_documents(docs)
retriever = vector_db.as_retriever()
# Create RAG agent
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
qa = RetrievalQA.from_chain_type(llm=llm,
                retriever=retriever)
# Ask a question
qa.invoke({"query": "What is an AI Agent?"})
```

@code.withnarayan

6. Sample Output

Q : What is an AI Agent?

A : An AI Agent is an autonomous system that uses an LLM, tools and memory to perceive, plan and take actions to achieve a goal.

Sources : notes.pdf (p. 3), article.com

7. Interview Revision (Important Questions)

1. What is an AI Agent ?
2. How is an AI Agent different from a chatbot ?
3. Explain the role of tools and function calling.
4. What is RAG and why is it used ?
5. What is the difference between LangChain and LangGraph ?
6. How do you add memory to an AI Agent ?
7. How do you evaluate an AI Agent ?

8. Key Takeaways

- AI Agents combine LLM + Tools + Memory + Planning.
- RAG makes agents more accurate and domain-specific.
- Frameworks like LangChain / LangGraph help build production-ready agents.
- Focus on evaluation, security and responsible AI.

9. Quick Revision – FAQs (One-Liners)

Topic	One-Liner Answer
AI Agent	An autonomous system that perceives, plans and takes actions.
Tools	External capabilities (e.g., web search, calculator, API).
Function Calling	Letting the LLM call predefined functions with structured input.
Memory	Stores conversation context (short-term or long-term).
RAG	Combines retrieval of external knowledge with LLM generation.
LangChain	Framework to build LLM apps and agents quickly.
LangGraph	Builds stateful, complex and multi-agent workflows.